# Problem Solving Methods

## Chapter 2

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Introduction

Problem solving, particularly in artificial intelligence, may be characterized as a systematic search through a range of possible actions in order to reach some predefined goal or solution.

Problem-solving methods divide into special purpose and general purpose.

A special-purpose method is tailor-made for a particular problem and often exploits very specific features of the situation in which the problem is embedded. In contrast, a general-purpose method is applicable to a wide variety of problems.

One general-purpose technique used in AI is means-end analysis—a step-by-step, or incremental, reduction of the difference between the current state and the final goal.

Available on notes.arpanadhikari7.com.np

Four general steps in problem solving:

– Goal formulation

  – What are the successful world states

– Problem formulation

  – What actions and states to consider given the goal

– Search

  – Determine the possible sequence of actions that lead to the states of known values and then choosing the best sequence.

– Execute

  – Give the solution perform the actions.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Problem formulation

A problem is defined by:

– An initial state: State from which agent start

– Successor function: Description of possible actions available to the agent.

– Goal test: Determine whether the given state is goal state or not

– Path cost: Sum of cost of each path from initial state to the given state.

A solution is a sequence of actions from initial to goal state. Optimal solution has the lowest path cost.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

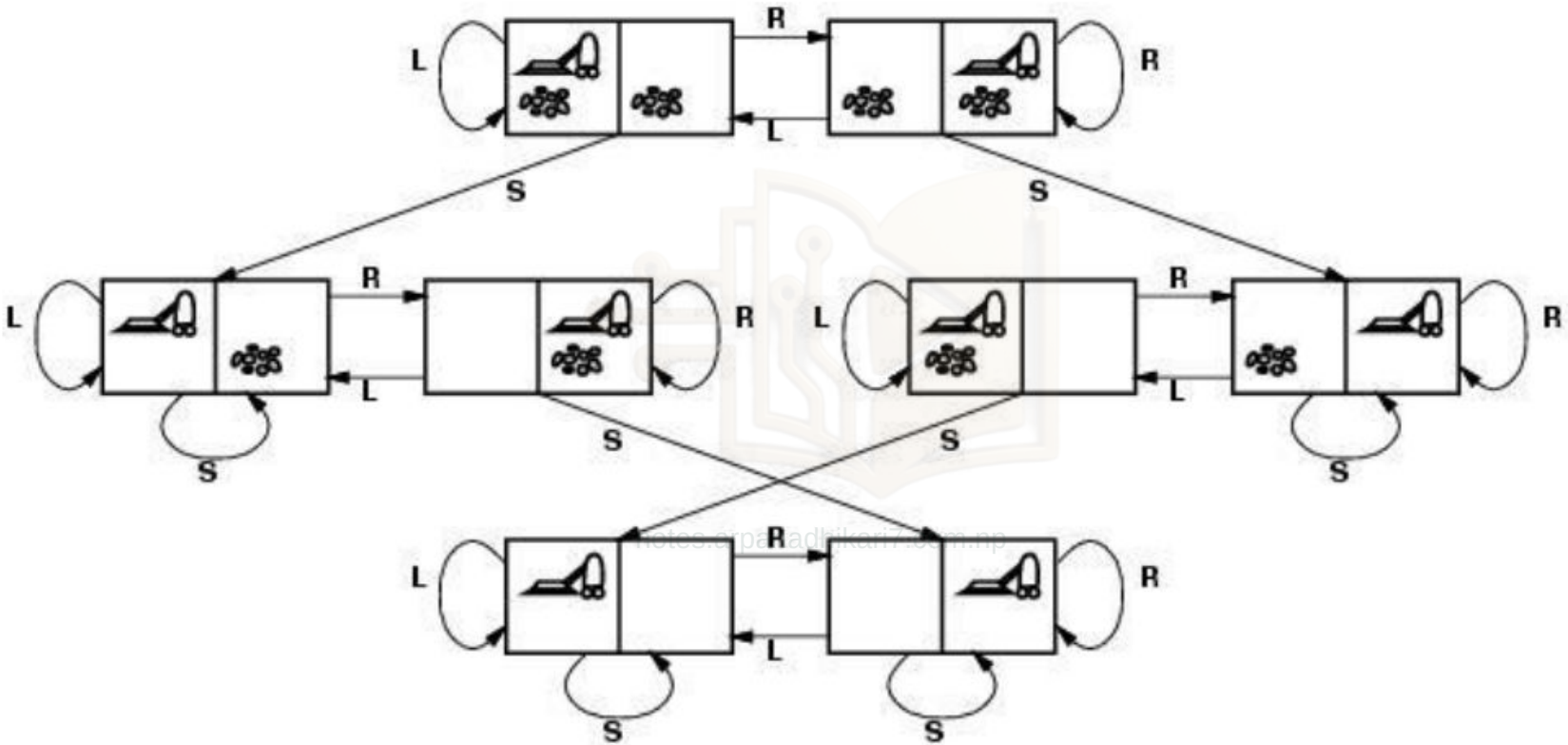
# State Space representation

The state space is commonly defined as a directed graph in which each node is a state and each arc represents the application of an operator transforming a state to a successor state.

A solution is a path from the initial state to a goal state.

notes.arpanadhikari7.com.np

# State Space representation of Vacuum World Problem:




Available on notes.arpanadhikari7.com.np

States?? two locations with or without dirt: 2 x 2^2=8 states.

Initial state?? Any state can be initial

Actions?? {Left, Right, Suck}

Goal test?? Check whether squares are clean.

Path cost?? Number of actions to reach goal.

Available on notes.arpanadhikari7.com.np

# Water leakage problem

If hall _wet and kitchen_dry

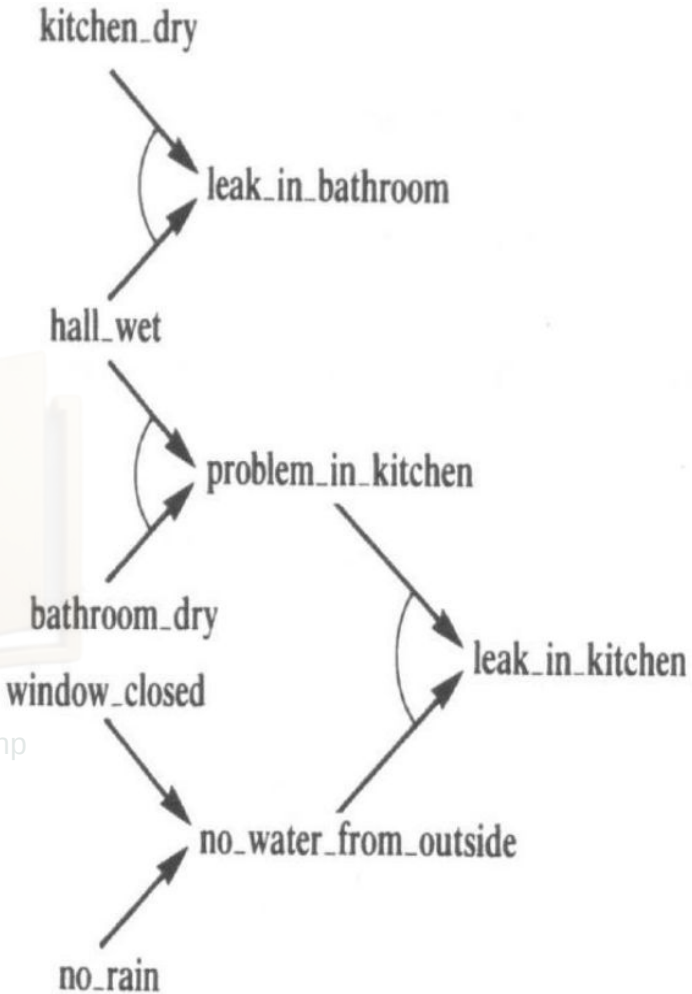
then leak_in_bathroom

If hall_wet and bathroom_dry

then problem_in_kitchen

If window_closed or no_rain

then no_water_from_outside




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Production System

A production system (or production rule system) is a computer program typically used to provide some form of artificial intelligence, which consists primarily of a set of rules about behavior.

These rules, termed productions, are a basic representation found useful in automated planning, expert systems and action selection.

A production system provides the mechanism necessary to execute productions in order to achieve some goal for the system.

tes.arpanadhikari7.com.np

Productions consist of two parts: a sensory precondition (or "IF" statement) and an action (or "THEN").

If a production's precondition matches the current state of the world, then the production is said to be triggered. If a production's action is executed, it is said to have fired.

A production system also contains a database, sometimes called working memory, which maintains data about current state or knowledge, and a rule interpreter.

The rule interpreter must provide a mechanism for prioritizing productions when more than one is triggered.

notes.arpanadhikari7.com.np

The underlying idea of production systems is to represent knowledge in the form of condition-action pairs called production rules:

If the condition C is satisfied then the action A is appropriate.

Available on notes.arpanadhikari7.com.np

# Types of production rules

Situation-action rules

If it is raining then open the umbrella.

Inference rules

If Cesar is a man then Cesar is a person

Production system is also called ruled-based system

Architecture of Production System:

Short Term Memory:

- Contains the description of the current state.

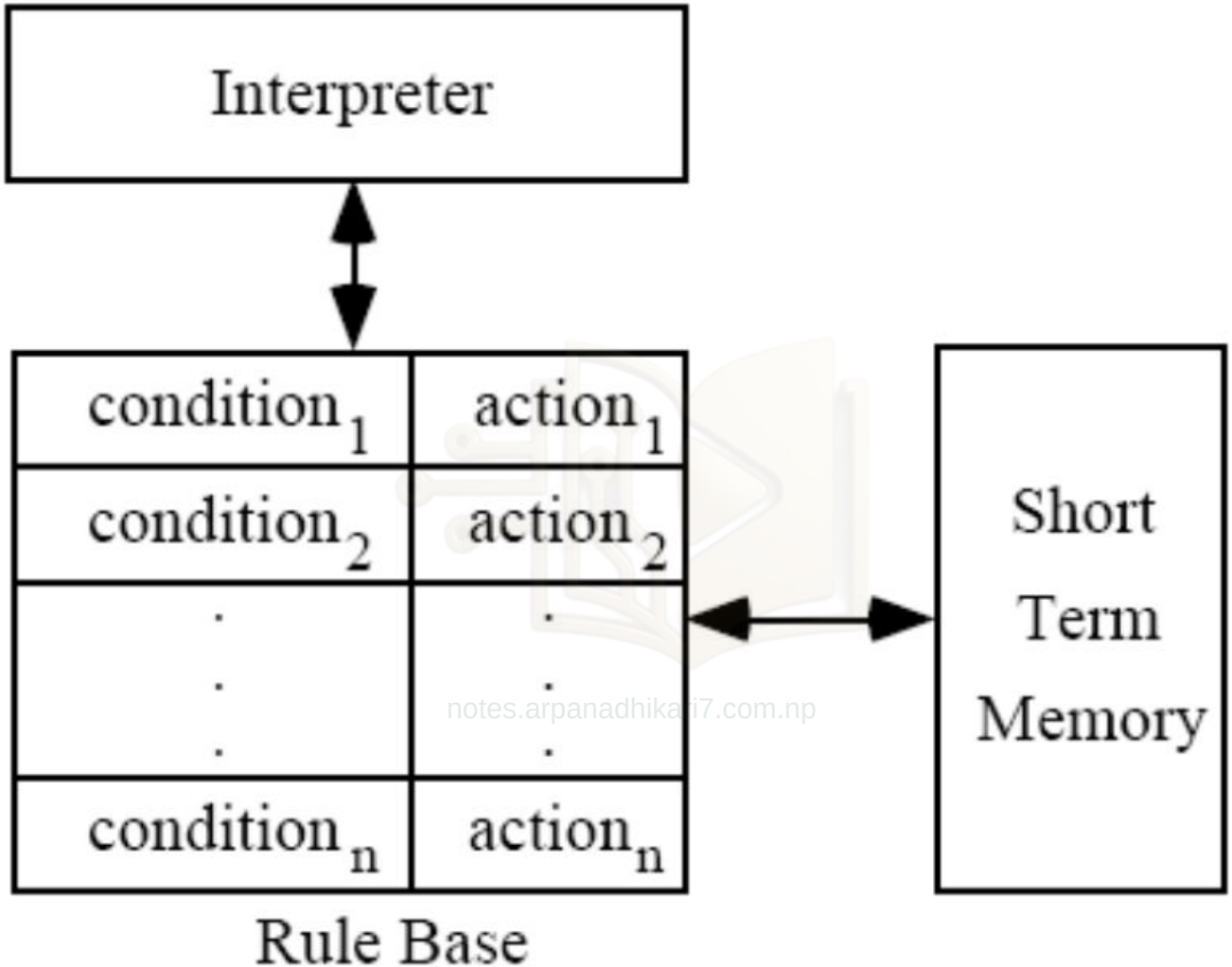
Set of Production Rules:

- Set of condition-action pairs and defines a single chunk of problem solving knowledge.

Interpreter:

- A mechanism to examine the short term memory and to determine which rules to fire (According to some strategies such as DFS, BFS, Priority, first-encounter etc)

notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The execution of a production system can be defined as a series of recognize-act cycles:

Match –memory contain matched against condition of production rules, this produces a subset of production called conflict set.

Conflict resolution –one of the production in the conflict set is then selected, Apply the rule.

Consider an example:

Problem: Sorting a string composed of letters a, b & c.

notes.arpanadhikari7.com.np

Short Term Memory: cbaca

Production Set:

1. ba → ab
2. ca → ac.
3. cb → bc

Interpreter: Choose one rule according to some strategy.

| Iteration # | Memory | Conflict Set | Rule fired |
|---|---|---|---|
| 0 | cbaca | 1, 2, 3 | 1 |
| 1 | cabca | 2 | 2 |
| 2 | acbca | 2, 3 | 2 |
| 3 | acbac | 1, 3 | 1 |
| 4 | acabc | 2 | 2 |
| 5 | aacbc | 3 | 3 |
| 6 | aabcc | ø | halt |

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Production System: The water jug problem

Problem: There are two jugs, a 4-gallon one and a 3-gallon one. Neither jug has any measuring markers on it. There is a pump that can be used to fill the jugs with water. How can you get exactly n ( 0, 1, 2, 3, 4) gallons of water into one of the two jugs ?

Solution Paradigm:

- build a simple production system for solving this problem.

- represent the problem by using the state space paradigm.

State = (x, y); where: x represents the number of gallons in the 4-gallon jug; y represents the number of gallons in the 3-gallon jug. x ε{0, 1, 2, 3, 4} and y ε{0, 1, 2, 3}.

The initial state represents the initial content of the two jugs. For instance, it may be (2, 3), meaning that the 4-gallon jug contains 2 gallons of water and the 3-gallon jug contains three gallons of water.

notes.arpanadhikari7.com.np

The goal state is the desired content of the two jugs.

The left hand side of a production rule indicates the state in which the rule is applicable and the right hand side indicates the state resulting after the application of the rule.

Available on notes.arpanadhikari7.com.np

For instance;

(x, y) such that $x < 4 \rightarrow (4, y)$ represents the production

If the 4-gallon jug is not full then fill it from the pump.

## The rule base contains the following production rules:

1. (x, y) such that $x < 4 \rightarrow (4, y)$ ; Fill the 4-gallon jug from pump
2. (x, y) such that $y < 3 \rightarrow (x, 3)$ ; Fill the 3-gallon jug from pump
3. (x, y) such that $x > 0 \rightarrow (0, y)$ ; Empty the 4-gallon jug on the ground
4. (x, y) such that $y > 0 \rightarrow (x, 0)$ ; Empty the 3-gallon jug on the ground
5. (x, y) such that $x + y \geq 4, x < 4, y > 0 \rightarrow (4, y - (4 - x))$
   ; Completely fill the 4-gallon jug from the 3-gallon jug
6. (x, y) such that $x + y \geq 3, x > 0, y < 3 \rightarrow (x - (3 - y), 3)$
   ; Completely fill the 3-gallon jug from the 4-gallon jug
7. (x, y) such that $x + y \leq 4, y > 0 \rightarrow (x+y, 0)$
   ; Empty the 3-gallon jug into the 4-gallon jug
8. (x, y) such that $x + y \leq 3, x > 0 \rightarrow (0, x + y)$
   ; Empty the 4-gallon jug into the 3-gallon jug

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The short term memory contains the current state (x, y).

*Let us consider the initial situation (0, 0) and the goal situation (n, 2)*

short term memory : (0, 0)

1. Match: 1, 2. 2. Conflict resolution: select rule 2. 3. Apply the rule

short term memory becomes (0, 3)

1. Match: 1, 4, 7 2. Conflict resolution: select rule 7 3. Apply the rule

short term memory becomes (3, 0)

1. Match:1, 2, 3, 6, 8 2. Conflict resolution: select rule 2 3. Apply the rule

short term memory becomes (3, 3)

1. Match: 1, 3, 4, 5 2. Conflict resolution: select rule 5 3. Apply the rule

short term memory becomes (4, 2) Goal achieved

Available on notes.arpanadhikari7.com.np

# Solution:

The sequence of the applied rules:

Fill the 3-gallon jug from pump

Empty the 3-gallon jug into the 4-gallon jug

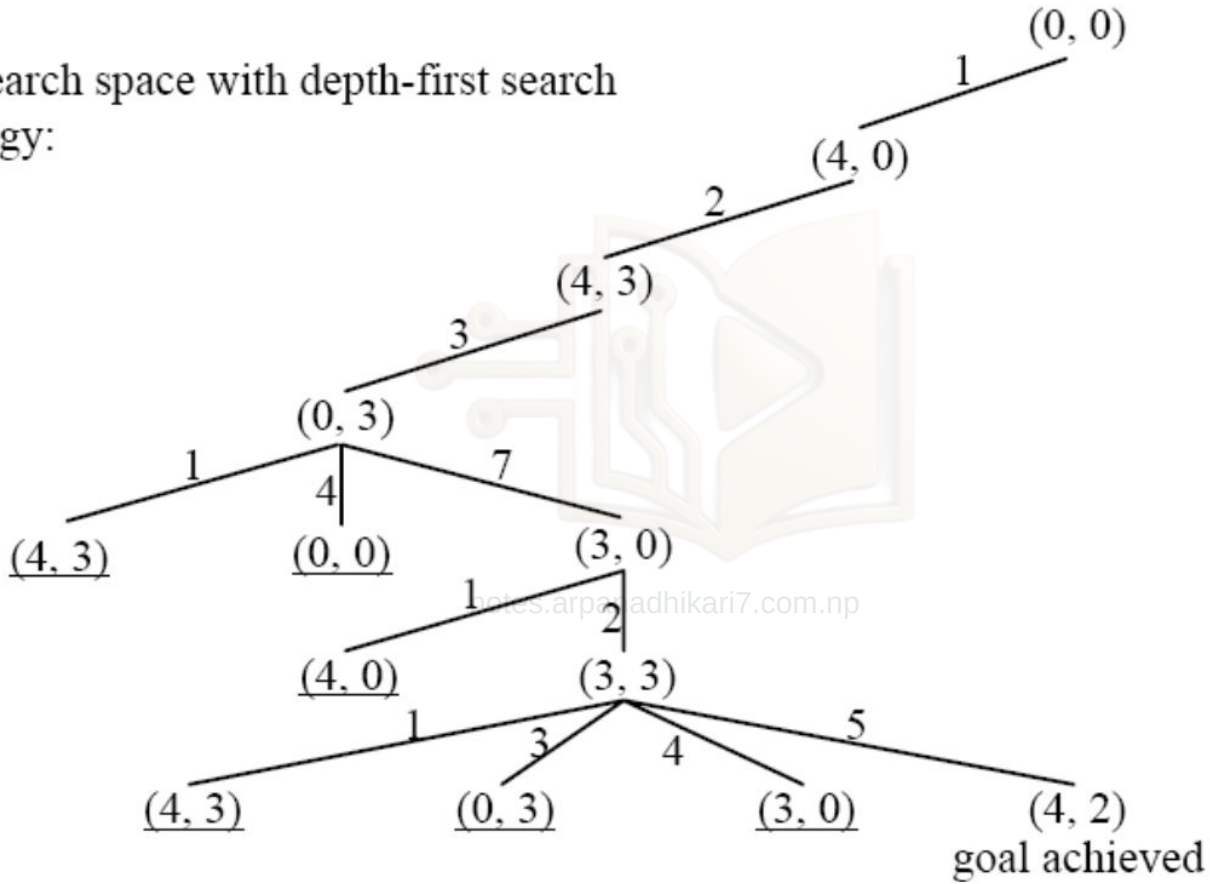
Fill the 3-gallon jug from pump

Fill the 4-gallon jug from the 3-gallon jug

notes.arpanadhikari7.com.np

## The Water Jug Problem: Representation

the search space with depth-first search strategy:




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Constraint Satisfaction Problem

A Constraint Satisfaction Problem is characterized by:

- a set of variables {x1, x2, .., xn},
- for each variable xi a domain Di with the possible values for that variable, and
- a set of constraints, i.e. relations, that are assumed to hold between the values of the variables. [These relations can be given intentionally, i.e. as a formula, or extensionally, i.e. as a set, or procedurally, i.e. with an appropriate generating or recognizing function.] We will only consider constraints involving one or two variables.

The constraint satisfaction problem is to find, for each i from 1 to n, a value in Di for xi so that all constraints are satisfied. Means that, we must find a value for each of the variables that satisfies all of the constraints.

A CS problem can easily be stated as a sentence in first order logic, of the form:

(exist x1)..(exist xn) (D1(x1) & .. Dn(xn) => C1..Cm)

Available on notes.arpanadhikari7.com.np

A CS problem is usually represented as an undirected graph, called Constraint Graph where the nodes are the variables and the edges are the binary constraints. Unary constraints can be disposed of by just redefining the domains to contain only the values that satisfy all the unary constraints. Higher order constraints are represented by hyperarcs.

In the following we restrict our attention to the case of unary and binary constraints.

Formally, a constraint satisfaction problem is defined as a triple $\langle X, D, C \rangle$, where $X$ is a set of variables, $D$ is a domain of values, and $C$ is a set of constraints. Every constraint is in turn a pair $\langle t, R \rangle$, where $t$ is a tuple of variables and $R$ is a set of tuples of values; all these tuples having the same number of elements; as a result $R$ is a relation. An evaluation of the variables is a function from variables to values, $v : X \rightarrow D$. Such an evaluation satisfies a constraint $\langle (x_1, \ldots, x_n), R \rangle$ if $(v(x_1), \ldots, v(x_n)) \in R$. A solution is an evaluation that satisfies all constraints.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Constraints

A constraint is a relation between a local collection of variables.

The constraint restricts the values that these variables can simultaneously have.

For example, all-diff(X1, X2, X3). This constraint says that X1, X2, and X3 must take on different values. Say that {1,2,3} is the set of values for each of these variables then:

X1=1, X2=2, X3=3 OK X1=1, X2=1,X3=3 NO

Available on notes.arpanadhikari7.com.np

The constraints are the key component in expressing a problem as a CSP.

The constraints are determined by how the variables and the set of values are chosen.

Each constraint consists of;

1. A set of variables it is over.
2. A specification of the sets of assignments to those variables that satisfy the constraint.

The idea is that we break the problem up into a set of distinct conditions each of which have to be satisfied for the problem to be solved.

Available on notes.arpanadhikari7.com.np

Example: In N-Queens: Place N queens on an N x N chess board so that queen can attack any other queen.

No queen can attack any other queen.

Given any two queens Qi and Qj they cannot attack each other.

Now we translate each of these individual conditions into a separate constraint.

Qi cannot attack Qj(i ≠j)

Qi is a queen to be placed in column i, Qj is a queen to be placed in column j.

The value of Qi and Qj are the rows the queens are to be placed in.

Note the translation is dependent on the representation we chose.

Queens can attack each other,

notes.arpanadhikari7.com.np

1. Vertically, if they are in the same column---this is impossible as Qi and Qj are placed in different columns.

2. Horizontally, if they are in the same row---we need the constraint Qi≠Qj.

Available on notes.arpanadhikari7.com.np

3. Along a diagonal, they cannot be the same number of columns apart as they are rows apart: we need the constraint |i-j| ≠|Qi-Qj| ( | | is absolute value)

Representing the Constraints;

1. Between every pair of variables (Qi,Qj) (i ≠j), we have a constraint Cij.

2. For each Cij, an assignment of values to the variables Qi= A and Qj= B, satisfies this constraint if and only if;

A ≠B

| A-B| ≠|i-j|

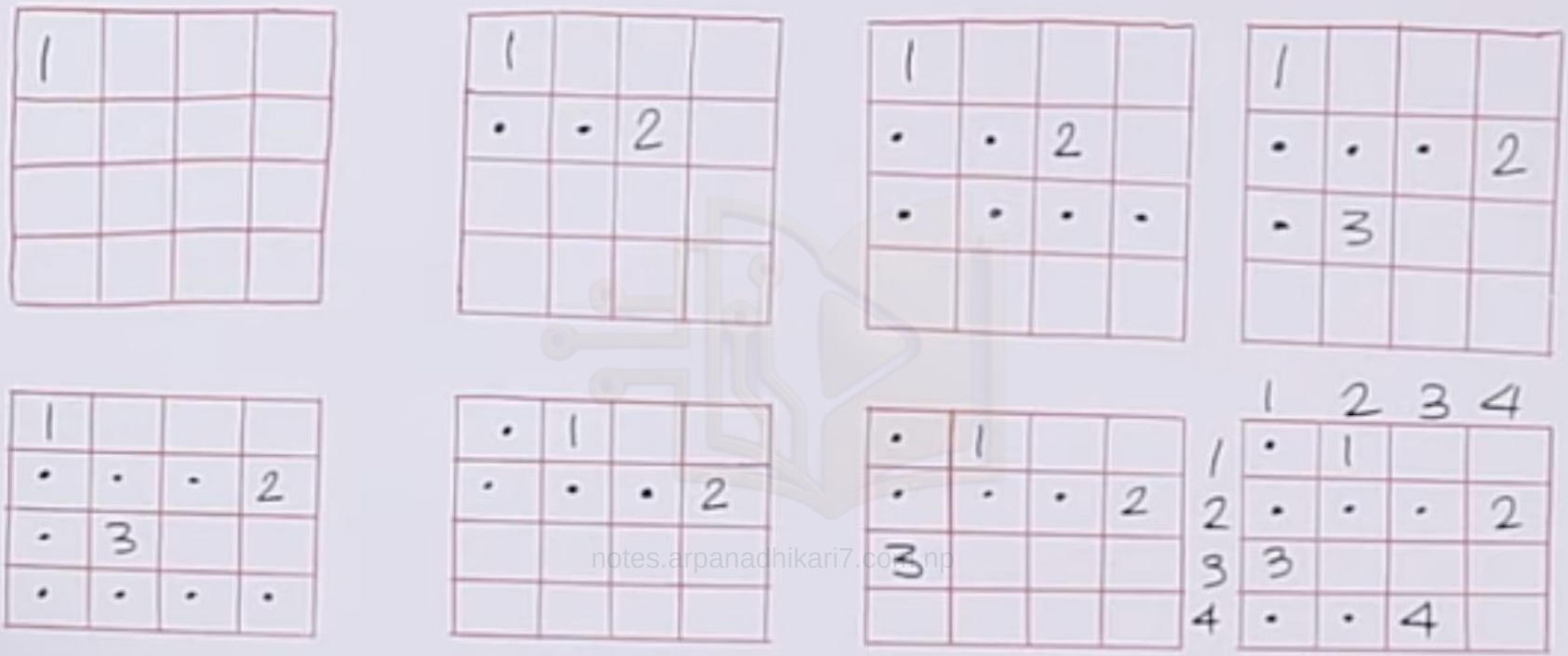
Solutions:

A solution to the N-Queens problem will be any assignment of values to the variables Q1,...,QN that satisfies all of the constraints.

Constraints can be over any collection of variables. In N-Queens we only need binary constraints---constraints over pairs of variables.

Available on notes.arpanadhikari7.com.np

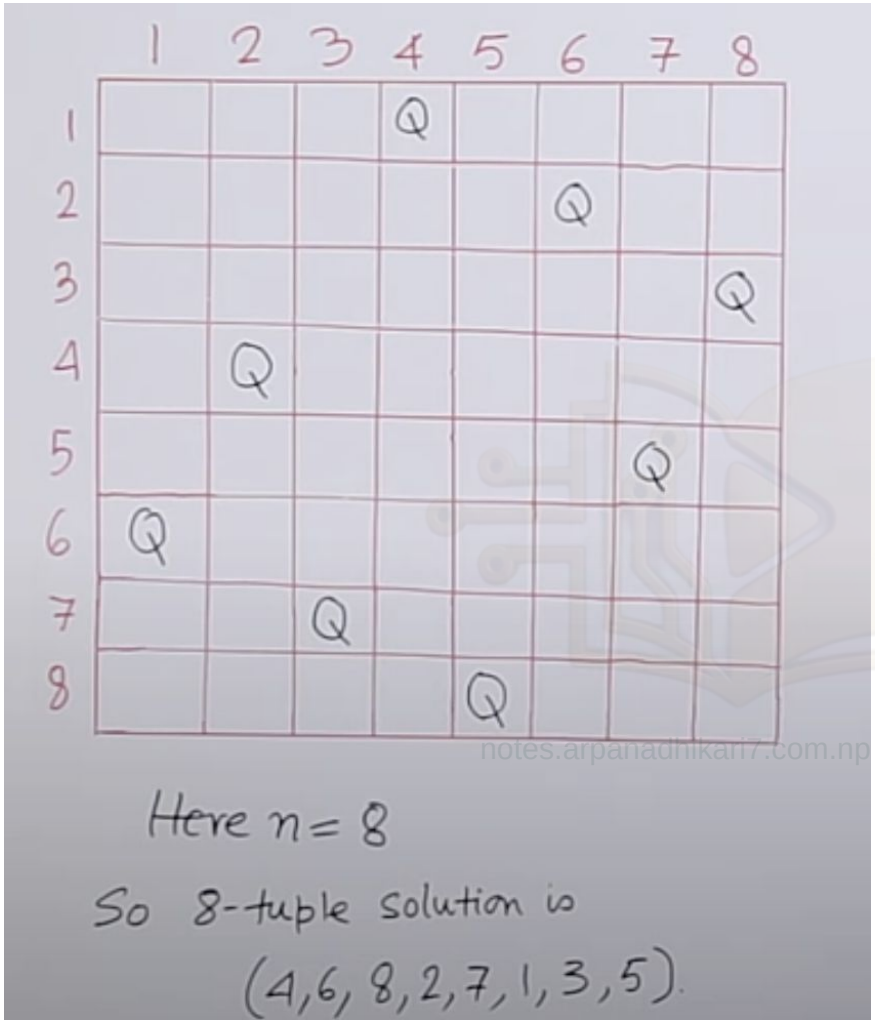
If n=4, then




4-tuple answer will be (2, 4, 1, 3).

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Cryptarithmetic problem

Cryptarithmetic Problem is a type of constraint satisfaction problem where the game is about digits and its unique replacement either with alphabets or other symbols.

In **cryptarithmetic problem,** the digits (0-9) get substituted by some possible alphabets or symbols.

The task in cryptarithmetic problem is to substitute each digit with an alphabet to get the result arithmetically correct.

notes.arpanadhikari7.com.np

We can perform all the arithmetic operations on a given cryptarithmetic problem.

**The rules or constraints on a cryptarithmetic problem are as follows:**

- There should be a unique digit to be replaced with a unique alphabet.
- The result should satisfy the predefined arithmetic rules, i.e., 2+2 =4, nothing else.
- Digits should be from **0-9** only.
- There should be only one carry forward, while performing the addition operation on a problem.
- The problem can be solved from both sides, i.e., **lefthand side (L.H.S), or righthand side (R.H.S)**

notes.arpanadhikari7.com.np

**Example: TO + GO = OUT**

- Given a cryptarithmetic problem, i.e., **S E N D + M O R E = M O N E Y**




notes.arpanadhikari7.com.np

**Follow the below steps to understand the given problem by breaking it into its subparts:**

- Starting from the left hand side (L.H.S) , the terms are **S** and **M**. Assign a digit which could give a satisfactory result. Let's assign **S->9** and **M->1**.




notes.arpanadhikari7.com.np

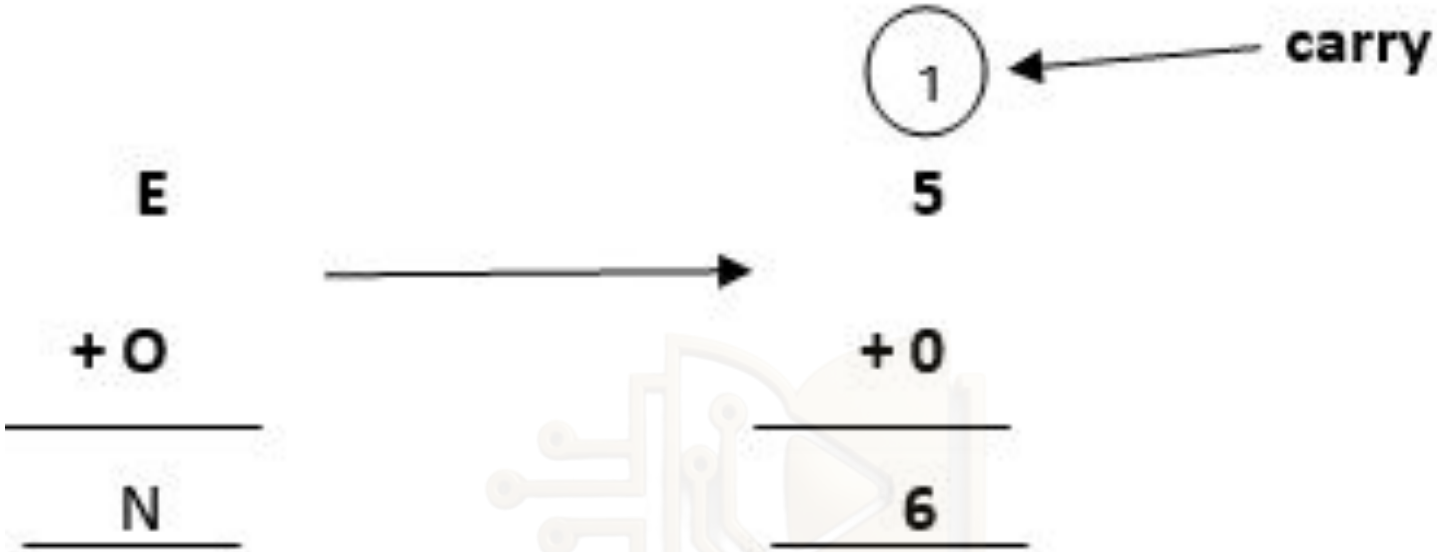
Hence, we get a satisfactory result by adding up the terms and got an assignment for **O** as **O->0** as well.

- Now, move ahead to the next terms **E** and **O** to get **N** as its output.




**Adding E and O, which means 5+0=5, which is not possible because** according to cryptarithmetic constraints, we cannot assign the same digit to two letters. So, we need to think more and assign some other value.

notes.arpanadhikari7.com.np

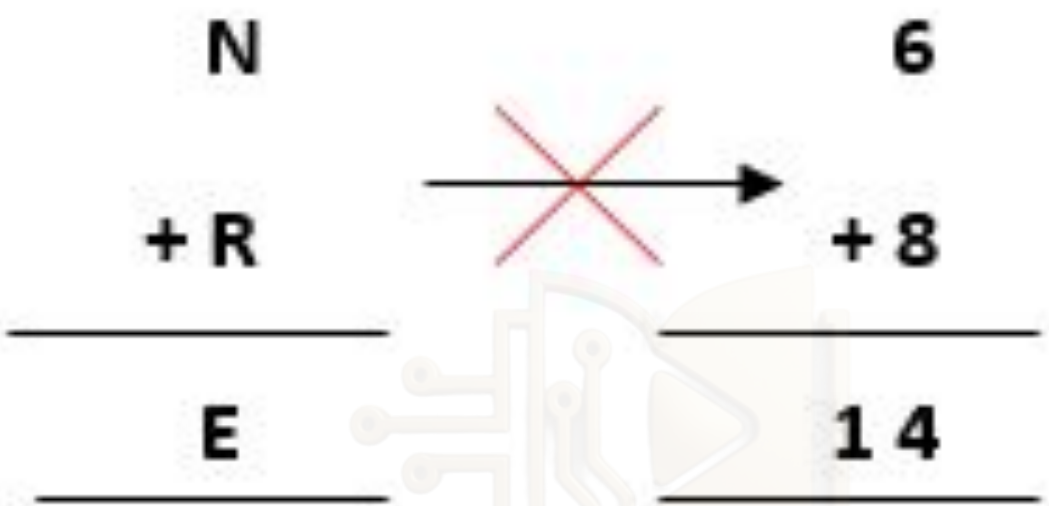



notes.arpanadhikari7.com.np

**Note: When we will solve further, we will get one carry, so after applying it, the answer will be satisfied.**

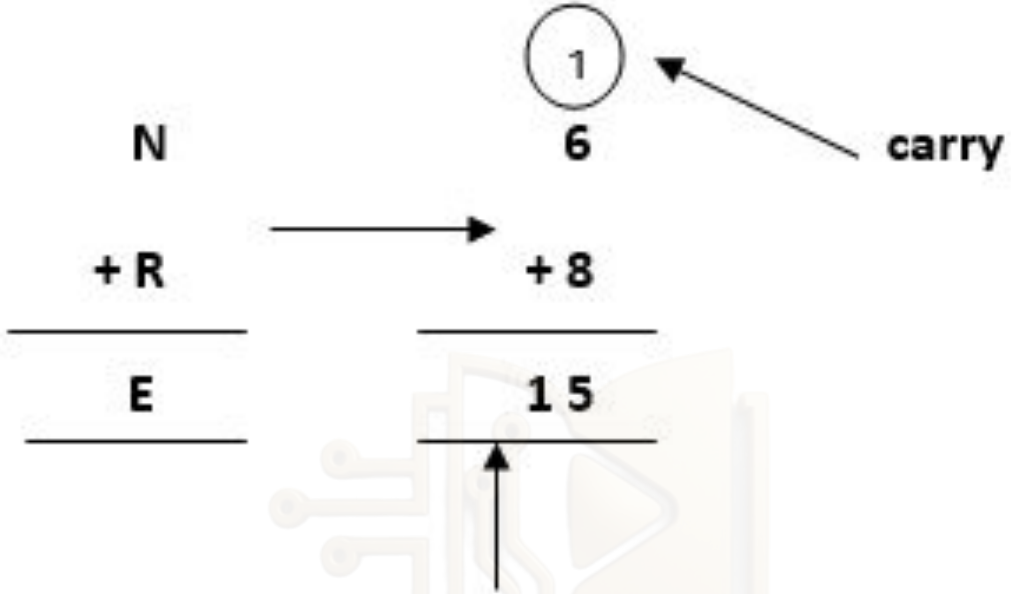
- Further, adding the next two terms **N** and **R** we get,

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

But, we have already assigned **E->5**. Thus, the above result does not satisfy the values
because we are getting a different value for **E.** So, we need to think more.
**Again, after solving the whole problem, we will get a carryover on this term, so our answer will be satisfied.**




**where 1 will be carry forward to the above term**

notes.arpanadhikari7.com.np

Let's move ahead.

- Again, on adding the last two terms, i.e., the rightmost terms **D** and **E**, we get **Y** as its result.




**where 1 will be  carry forward to the above term**

- Keeping all the constraints in mind, the final resultant is as follows:

notes.arpanadhikari7.com.np

- Below is the representation of the assignment of the digits to the alphabets.

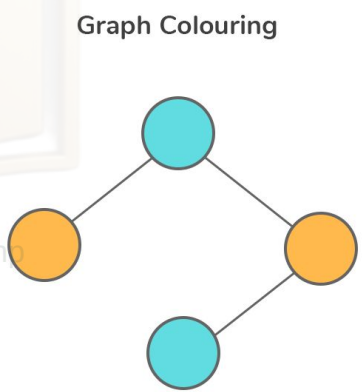
| | |
|---|---|
| S | 9 |
| E | 5 |
| N | 6 |
| D | 7 |
| M | 1 |
| O | 0 |
| R | 8 |
| y | 2 |

notes.arpanadhikari7.com.np

# Graph Coloring Problem

Graph coloring problem involves assigning colors to certain elements of a graph subject to certain restrictions and constraints. In other words, the process of assigning colors to the vertices such that no two adjacent vertexes have the same color is caller Graph Colouring.

This is also known as vertex coloring.




notes.arpanadhikari7.com.np

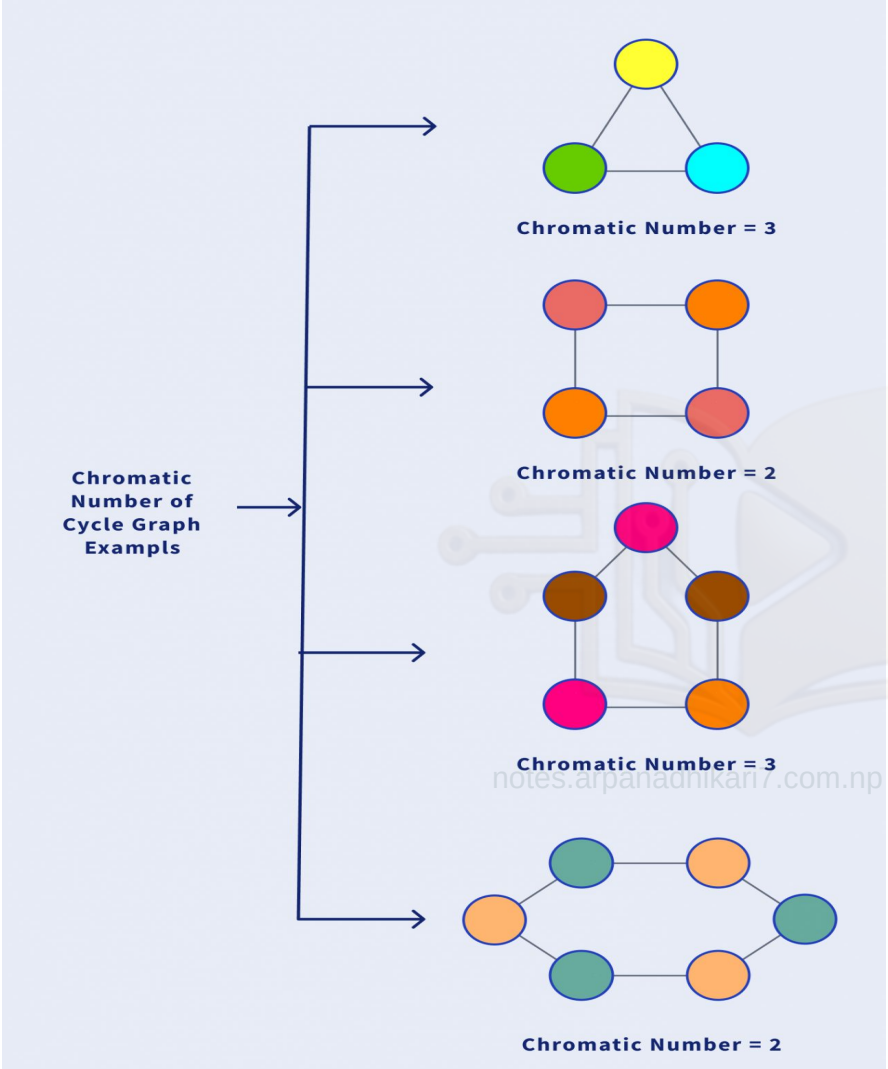
Chromatic Number:  The smallest number of colours needed to colour a graph G is called its chromatic number.

For example, in the above image, vertices can be coloured using a minimum of 2 colours.

Hence the chromatic number of the graph is 2.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

## Applications of Graph Colouring:

- Map Coloring
- Scheduling the tasks
- Preparing Time Table
- Assignment
- Conflict Resolution
- Sudoku

notes.arpanadhikari7.com.np

## Approach 1: Brute Force

- The simplest approach to solve this problem would be to generate all possible combinations (or configurations) of colours.
- After generating a configuration, check if the adjacent vertices have the same colour or not. If the conditions are met, add the combination to the result and break the loop.
- Since each node can be coloured by using any of the M colours, the total number of possible colour configurations are mV. The complexity is exponential which is very huge.

Available on notes.arpanadhikari7.com.np

## Approach 2: Backtracking

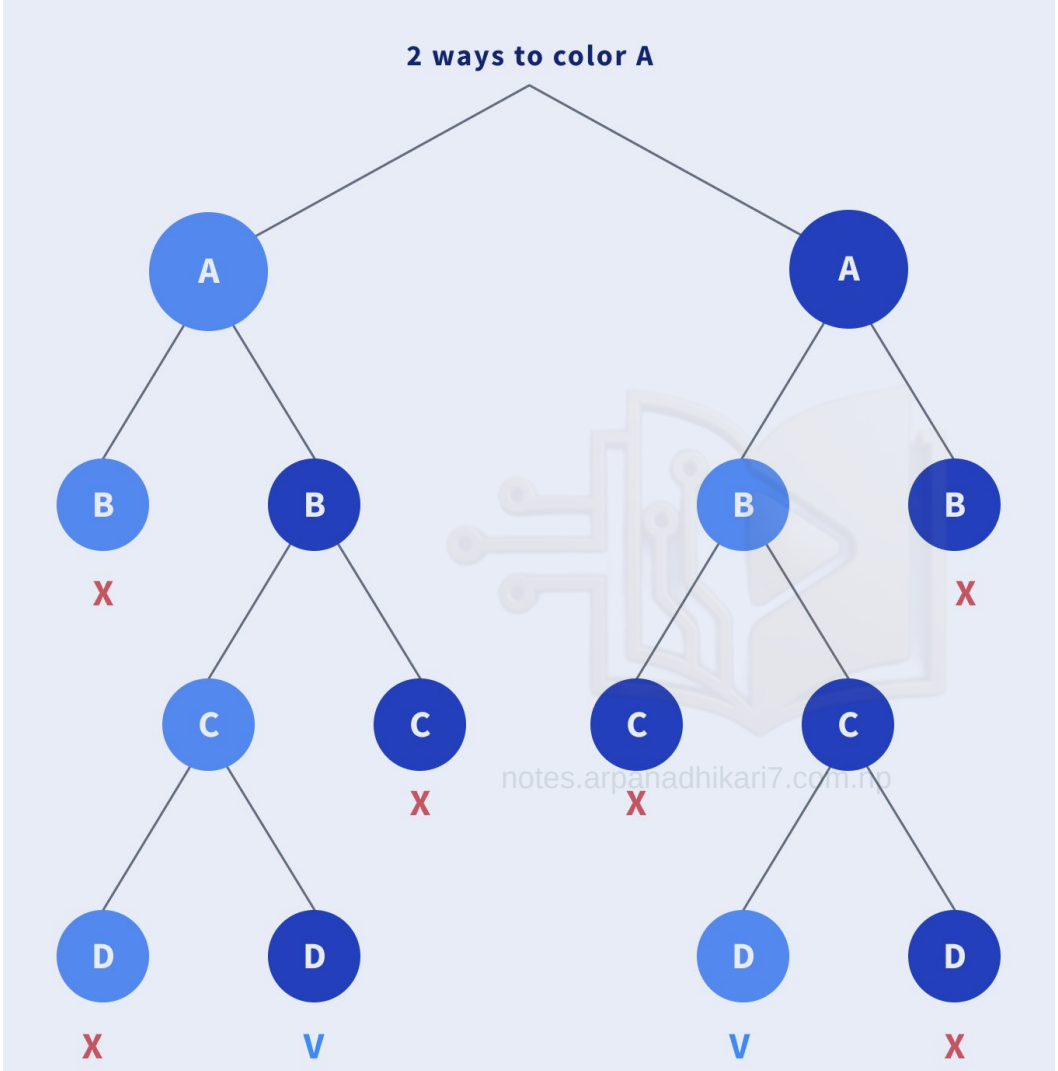
In the previous approach, trying and checking every possible combination was tedious and had an exponential time complexity.

Some of the permutation calculations were unnecessary but were calculated again and again. Therefore, the idea is to use a backtracking approach to solve the problem.

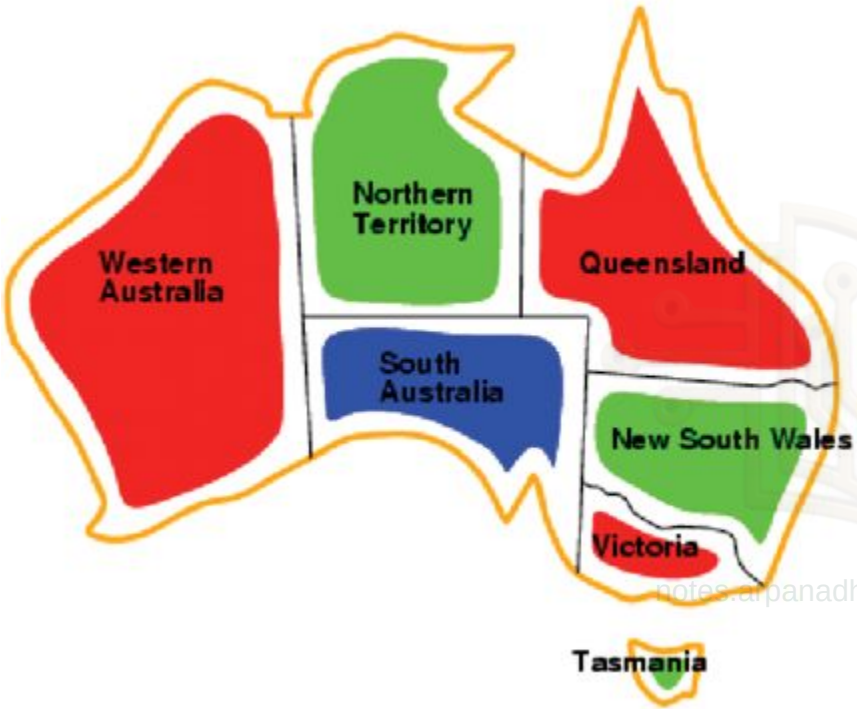
notes.arpanadhikari7.com.np

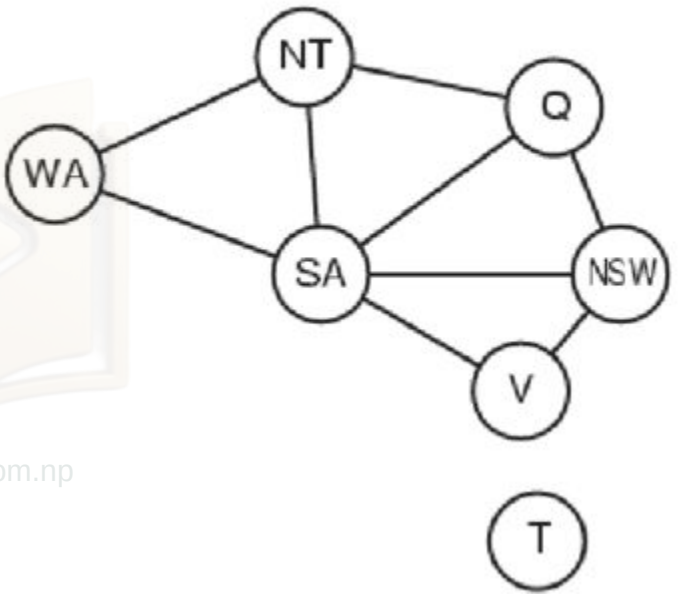
In this approach, the idea is to color a vertex and while coloring any adjacent vertex, choose a different color. Similarly, color every possible vertex following the restrictions, till any further vertex is left coloring. In any case, if all adjacent vertices for a given vertex are colored, then backtrack and change color.

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Searching

Figure below contains a representation of a map. The nodes represent cities, and the links represent direct road connections between cities. The number associated to a link represents the length of the corresponding road.

The search problem is to find a path from a city S to a city G

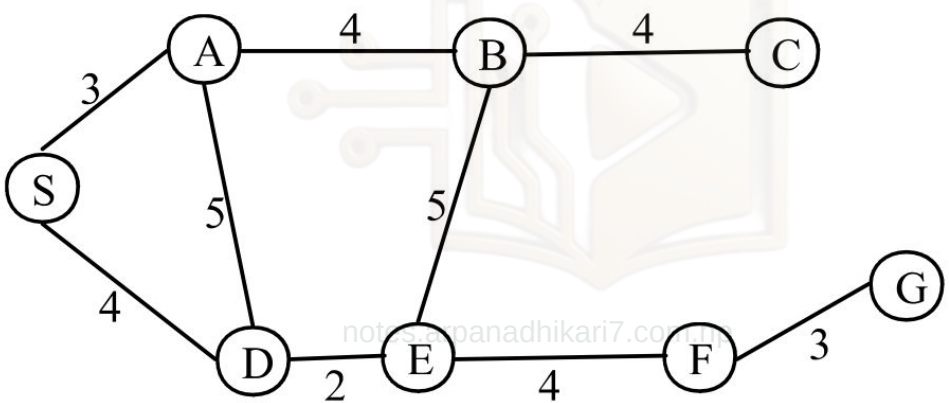



Figure : A graph representation of a map

This problem will be used to illustrate some search methods.

Available on notes.arpanadhikari7.com.np

Search problems are part of a large number of real world applications:

- VLSI layout
- Path planning
- Robot navigation etc.

There are two broad classes of search methods:

- **uninformed (or blind) search methods;**
- **heuristically informed search methods.**

**In the case of the uninformed search methods**, the order in which potential solution paths are considered is arbitrary, using no domain-specific information to judge where the solution is likely to lie.

notes.arpanadhikari7.com.np

**In the case of the heuristically informed search methods**, one uses domain-dependent (heuristic) information in order to search the space more efficiently.

Available on notes.arpanadhikari7.com.np

# Measuring problem Solving Performance

The performance of a search algorithm can be evaluated in four ways:

- Completeness: An algorithm is said to be complete if it definitely finds solution to the problem, if exist.
- Time Complexity: How long (worst or average case) does it take to find a solution? Usually measured in terms of the number of nodes explained.
- Space Complexity: How much space is used by the algorithm? Usually measured in terms of the maximum number of nodes in memory at a time.
- Optimality/ Admissibility: If a solution is found, is it guaranteed to be a optimal one? For example, is it the one with the minimum cost?

Time and space complexity is measured in terms of

b - maximum branching factor (number of successor of any node) of the search tree

d - depth of least cost solution

m - maximum length of any path in the space

notes.arpanadhikari7.com.np

# State space search

In Artificial Intelligence a state space consists of the following elements,

1. A (possibly infinite) set of states

1.1. Out of the possible states, one state represents the start state that is the initial state of the problem.

1.2. Each state represents some configuration reachable from the start state

1.3. Out of the possible states, some states may be goal states (solutions)

2. A set of rules,

notes.arpanadhikari7.com.np

2.1. Applying a rule to the current state, transforms it to another or a new state in the state space

2.2 All operators may not be applicable to all states in the state space

State spaces are used extensively in Artificial Intelligence (AI) to represent and solve problems.

Available on notes.arpanadhikari7.com.np

# Uninformed Search Strategies

**Uninformed search** (or **blind search**)
– Strategies have no additional information about states beyond that provided in the problem definition

**Informed** (or **heuristic**) search
– Search strategies know whether one state is more promising than another

Uninformed strategies (defined by order in which nodes are expanded):

– Breadth-first search
– Uniform-cost search
– Depth-first search
– Depth-limited search
– Iterative deepening search – Bi directional search

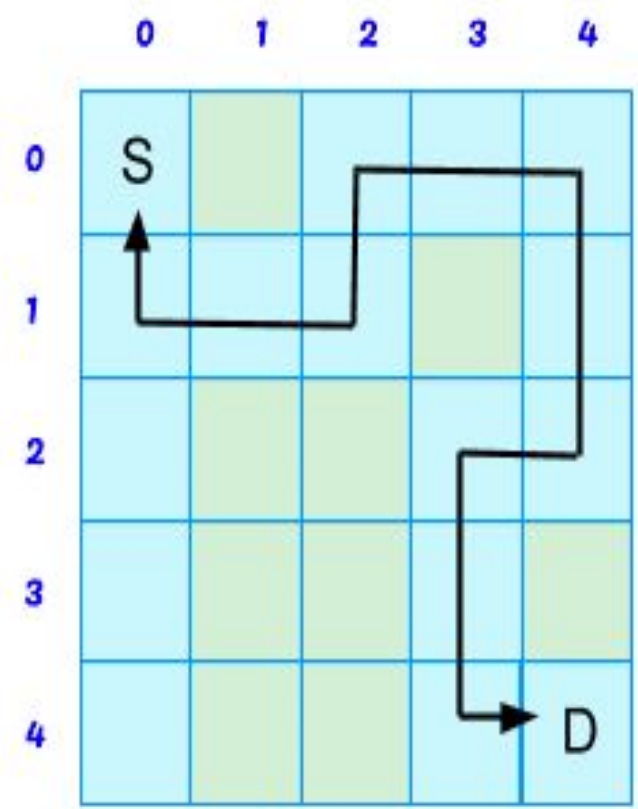
notes.arpanadhikari7.com.np

# State space search examples

## Example: Maze

A maze problem can be represented as a state-space

- Each state represents “where you are” that is the current position in the maze
- The start state or initial state represents your starting position
- The goal state represents the exit from the maze




Rules (for a rectangular maze) are: move north, move south, move east, and move west

- Each rule takes you to a new state (maze location)
- Rules may not always apply, because of walls in the maze

Available on notes.arpanadhikari7.com.np

# Example 2. The 15 Puzzle

Start state:

| | | | |
|---|---|---|---|
| 3 | 10 | 13 | 7 |
| 9 | 14 | 6 | 1 |
| 4 | | 15 | 2 |
| 11 | 8 | 5 | 12 |

Goal state:

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 |
| 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | |

The start state is some (almost) random configuration of the tiles and the goal state is as shown.

State Space Search Rules are

- Move empty space up
- Move empty space down
- Move empty space right
- Move empty space left

These Rules apply if empty space is not against the edge.

notes.arpanadhikari7.com.np

# General State space search problem

1. First, select some way to represent states in the given problem in an unambiguous way.

2. Next, formulate all actions or operators that can be performed in states, including their preconditions and effects.

- Actions or operates are called PRODUCTION RULES

3. Represent the initial state or states of the problem.

notes.arpanadhikari7.com.np

4. Formulate precisely when a state satisfies the goal of our problem.

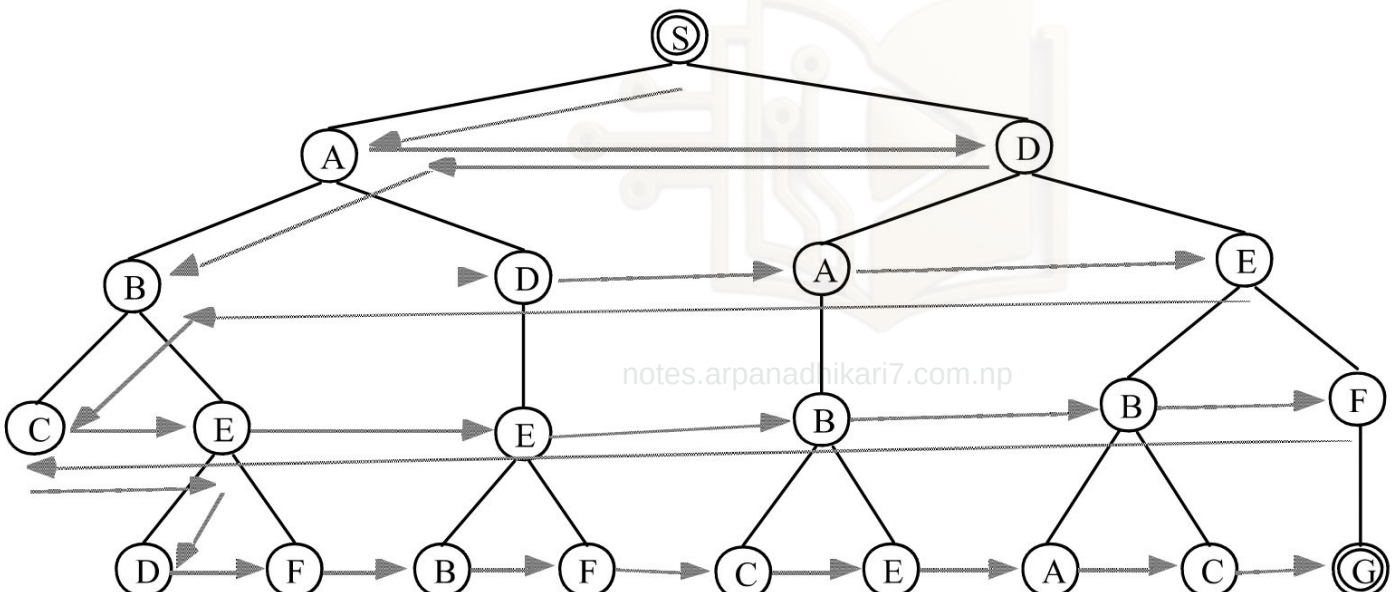
5. Activate the production rules on the initial state and its descendants, until a goal state is reached

## Breadth First Search

All nodes are expended at a given depth in the search tree before any nodes at the next level are expanded until the goal reached.

Expand *shallowest* unexpanded node. *fringe* is implemented as a FIFO queue

*Constraint: Do not generate as child node if the node is already parent to avoid more loop.*




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

**Fringe: it is the collection of nodes that have been generated but not yet expanded.**

**The set of all leaf nodes available for expansion at any given point is called the frontier.**

**Implementation: Fringe / Frontier is a FIFO queue.**

**BFS Evaluation:**

Completeness:

- *Does it always find a solution if one exists?*
- YES
  - If shallowest goal node is at some finite depth d and If $b$ is finite

Time complexity:

- Assume a state space where every state has $b$ successors.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

- root has $b$ successors, each node at the next level has again b successors (total $b^2$), …
- Assume solution is at depth $d$
- Worst case; expand all except the last node at depth $d$
- Total no. of nodes generated:

$$b + b^2 + b^3 + \ldots\ldots\ldots\ldots\ldots\ldots b^d + (b^{d+1} - b) = \mathbf{O(b^{d+1})}$$

Space complexity:

- Each node that is generated must remain in memory
- Total no. of nodes in memory:

$$1 + b + b^2 + b^3 + \ldots\ldots\ldots\ldots\ldots\ldots b^d + (b^{d+1} - b) = \mathbf{O(b^{d+1})}$$

Optimal (i.e., admissible):

- if all paths have the same cost. Otherwise, not optimal but finds solution with shortest path length (shallowest solution). If each path does not have same path cost shallowest solution may not be optimal

Two lessons:

- Memory requirements are a bigger problem than its execution time.
- Exponential complexity search problems cannot be solved by uninformed search methods for any but the smallest instances.

Available on notes.arpanadhikari7.com.np

| DEPTH2 | NODES | TIME | MEMORY |
|---|---|---|---|
| 2 | 1100 | 0.11 seconds | 1 megabyte |
| 4 | 111100 | 11 seconds | 106 megabytes |
| 6 | 107 | 19 minutes | 10 gigabytes |
| 8 | 109 | 31 hours | 1 terabyte |
| 10 | 1011 | 129 days | 101 terabytes |
| 12 | 1013 | 35 years | 10 petabytes |
| 14 | 1015 | 3523 years | 1 exabyte |

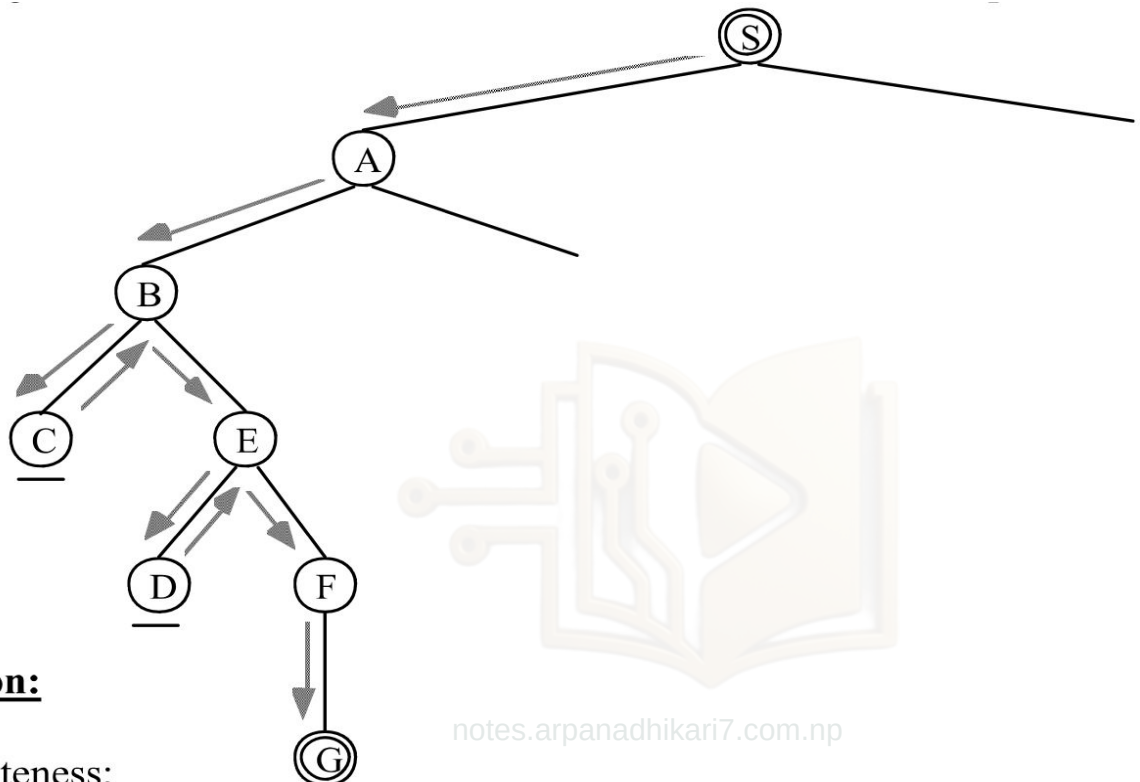
# Depth First Search

notes.arpanadhikari7.com.np

Looks for the goal node among all the children of the current node before using the sibling of this node i.e. expand deepest unexpanded node.

Here Fringe is implemented as Stack or (LIFO queue)

Available on notes.arpanadhikari7.com.np




**DFS Evaluation:**

notes.arpanadhikari7.com.np

Completeness;

- *Does it always find a solution if one exists?*
- NO
  - If search space is infinite and search space contains loops then DFS may not find solution.

Available on notes.arpanadhikari7.com.np

Time complexity;

- Let m is the maximum depth of the search tree. In the worst case Solution may exist at depth m.
- root has *b* successors, each node at the next level has again b successors (total $b^2$), …
- Worst case; expand all except the last node at depth *m*
- Total no. of nodes generated:

$$b + b^2 + b^3 + \ldots\ldots\ldots\ldots\ldots\ldots b^m = \mathbf{O(b^m)}$$

Space complexity:

- It needs to store only a single path from the root node to a leaf node, along with remaining unexpanded sibling nodes for each node on the path.
- Total no. of nodes in memory:

$$1 + b + b + b + \ldots\ldots\ldots\ldots\ldots\ldots \text{ b m times} = \mathbf{O(bm)}$$

notes.arpanadhikari7.com.np

Optimal (i.e., admissible):

- DFS expand deepest node first, if expands entire let sub-tree even if right sub-tree contains goal nodes at levels 2 or 3. Thus we can say DFS may not always give optimal solution.

Available on notes.arpanadhikari7.com.np

## Uniform Cost Search:

**Uniform-cost search** (**UCS**) is modified version of BFS to make optimal. It is basically a tree search algorithm used for traversing or searching a weighted tree, tree structure, or graph. The search begins at the root node. The search continues by visiting the next node which has the least total cost from the root. Nodes are visited in this manner until a goal state is reached.

Typically, the search algorithm involves expanding nodes by adding all unexpanded neighboring nodes that are connected by directed paths to a priority queue. In the queue, each node is associated with its total path cost from the root, where the least-cost paths are given highest priority. The node at the head of the queue is subsequently expanded, adding the next set of connected nodes with the total path cost from the root to the respective node. The uniform-cost search is **complete** and **optimal** if the cost of each step exceeds some positive bound ε.

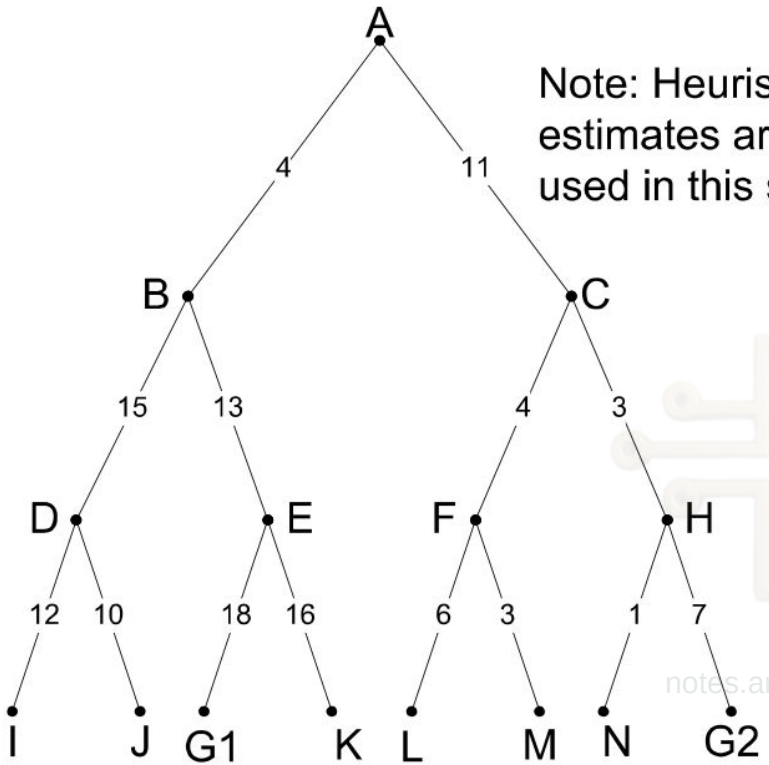
Does not care about the number of steps, only care about total cost.

notes.arpanadhikari7.com.np

•Complete? Yes, if step cost ≥ε (small positive number).
•Time? Maximum as of BFS
•Space? Maximum as of BFS.
•Optimal? Yes

**Consider an example:**

Available on notes.arpanadhikari7.com.np



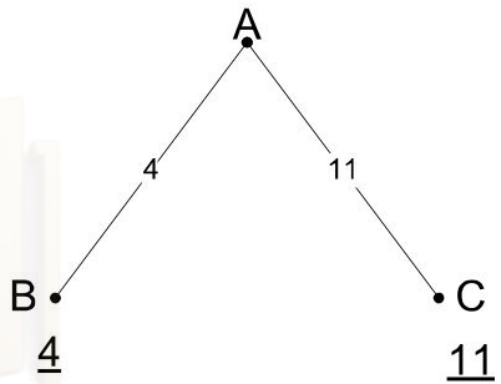

Note: Heurisitic estimates are not used in this search!

- Start with root node A.

A

0

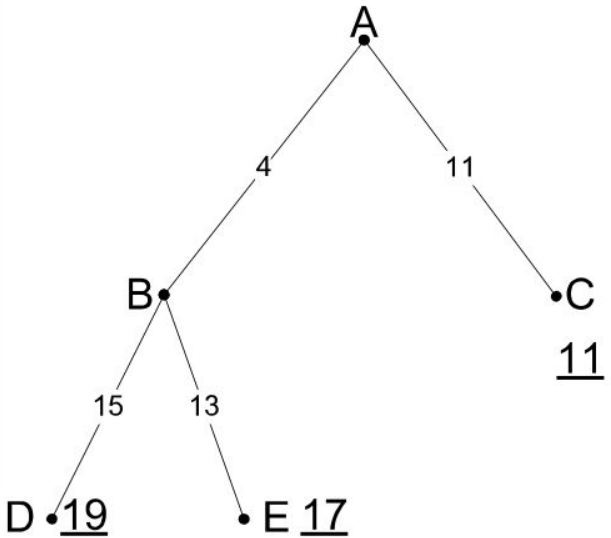
Paths from root are generated.



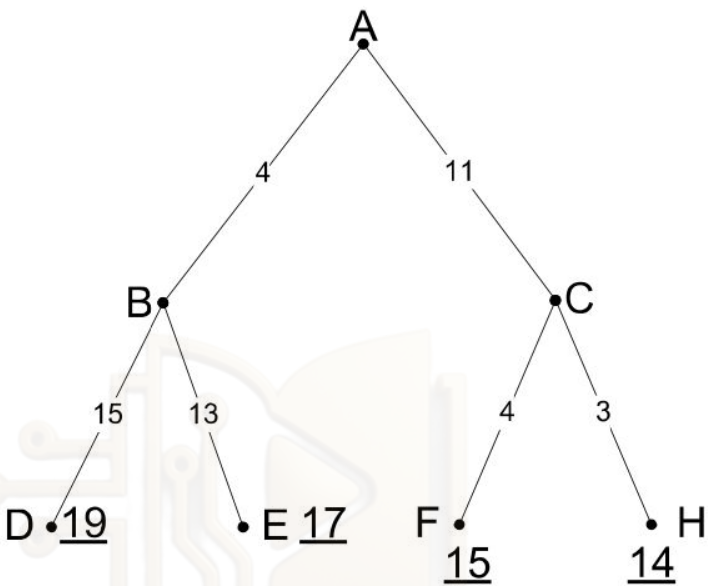

Since B has the least cost, we expand it.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np



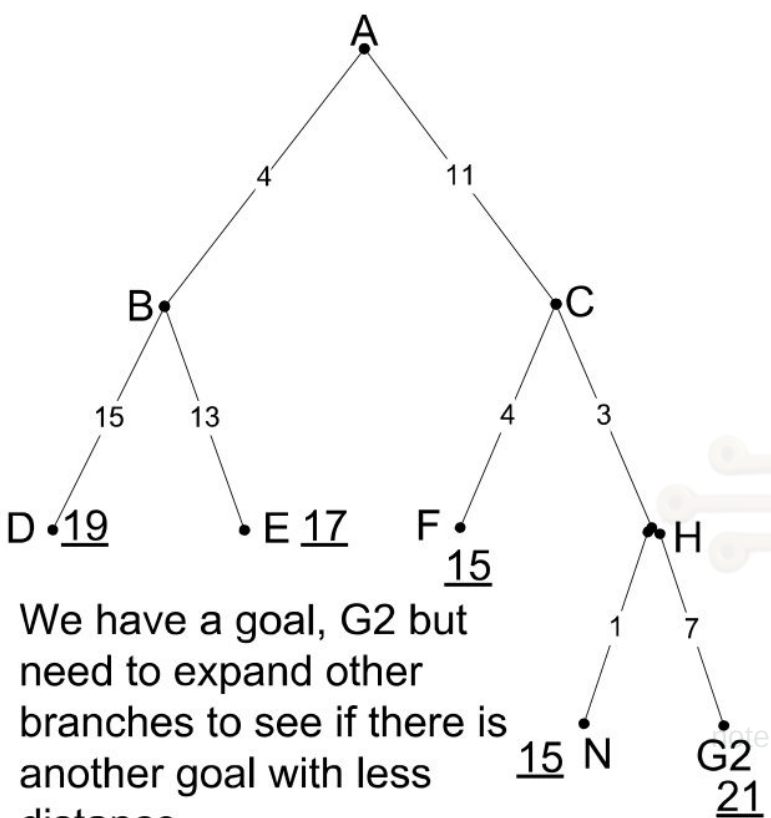

Of our 3 choices, C has the least cost so we'll expand it.



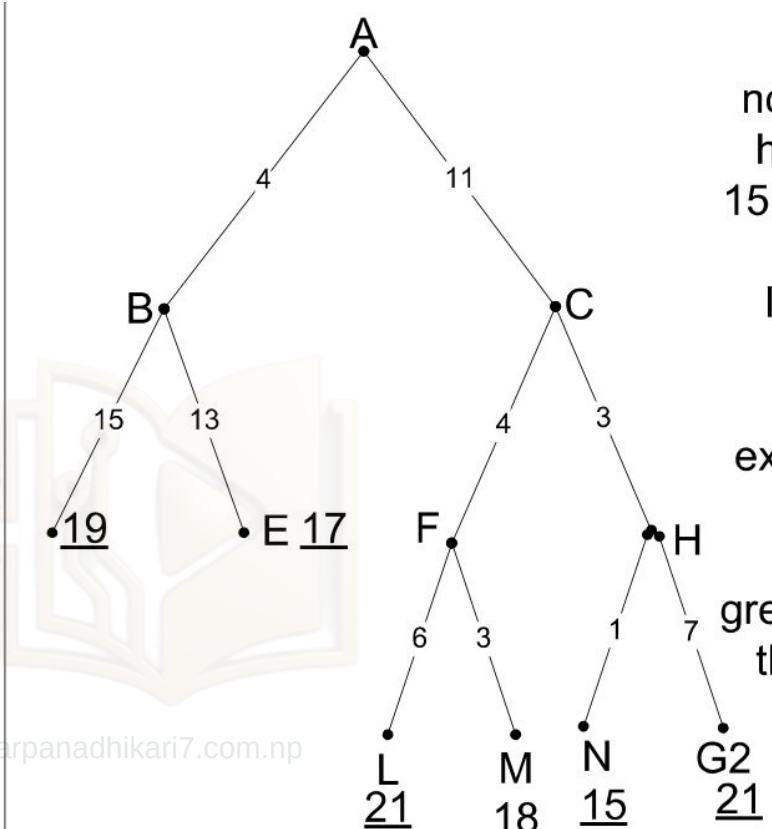

Node H has the least cost thus far, so we expand it.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




We have a goal, G2 but need to expand other branches to see if there is another goal with less distance.




Note: Both nodes F and N have a cost of 15, we chose to expand the leftmost node first. We continue expanding until all remaining paths are greater than 21, the cost of G2

Available on notes.arpanadhikari7.com.np

## Depth Limited Search:

The problem of unbounded trees can be solve by supplying depth-first search with a determined depth limit (nodes at depth are treated as they have no successors) –**Depth limited search. Depth-limited search** is an algorithm to explore the vertices of a graph. It is a modification of depth-first search and is used for example in the iterative deepening depth-first search algorithm.

Like the normal depth-first search, depth-limited search is an uninformed search. It works exactly like depth-first search, but avoids its drawbacks regarding completeness by imposing a maximum limit on the depth of the search. Even if the search could still expand a vertex beyond that depth, it will not do so and thereby it will not follow infinitely deep paths or get stuck in cycles. Therefore depth-limited search will find a solution if it is within the depth limit, which guarantees at least completeness on all graphs.

It solves the infinite-path problem of DFS. Yet it introduces another source of problem if we are unable to find good guess of *l*. Let d is the depth of shallowest solution.

If $l < d$ then incompleteness results.
If $l > d$ then not optimal.

Time complexity: $O(b^l)$
Space complexity: $O(bl)$

Available on notes.arpanadhikari7.com.np

## Iterative Deepening Depth First Search:

In this strategy, depth-limited search is run repeatedly, increasing the depth limit with each iteration until it reaches *d*, the depth of the shallowest goal state. On each iteration, IDDFS visits the nodes in the search tree in the same order as depth-first search, but the cumulative order in which nodes are first visited, assuming no pruning, is effectively breadth-first.

IDDFS combines depth-first search's space-efficiency and breadth-first search's completeness (when the branching factor is finite). It is optimal when the path cost is a non-decreasing function of the depth of the node.

The technique of *iterative deepening* is based on this idea. *Iterative deepening* is depth-first search to a fixed depth in the tree being searched. If no solution is found up to this depth then the depth to be searched is increased and the whole `bounded' depth-first search begun again.

notes.arpanadhikari7.com.np

IIt works by setting a depth of search -say, depth 1- and doing depth-first search to that depth. If a solution is found then the process stops -otherwise, increase the depth by, say, 1 and repeat until a solution is found. Note that every time we start up a new bounded depth search *we start from scratch* - i.e. we throw away any results from the previous search.

Available on notes.arpanadhikari7.com.np

Now *iterative deepening* is a popular method of search. We explain why this is so.

Depth-first search can be implemented to be much cheaper than breadth-first search in terms of memory usage -but it is not guaranteed to find a solution even where one is guaranteed.

On the other hand, breadth-first search can be guaranteed to terminate if there is a winning state to be found and will always find the `quickest' solution (in terms of how many steps need to be taken from the root node). It is, however, a very expensive method in terms of memory usage.

*Iterative deepening* is liked because it is an effective compromise between the two other methods of search. It is a form of depth-first search with a lower bound on how deep the search can go. Iterative deepening terminates if there is a solution. It can produce the same solution that breadth-first search would produce but does not require the same memory usage (as for breadth-first search).

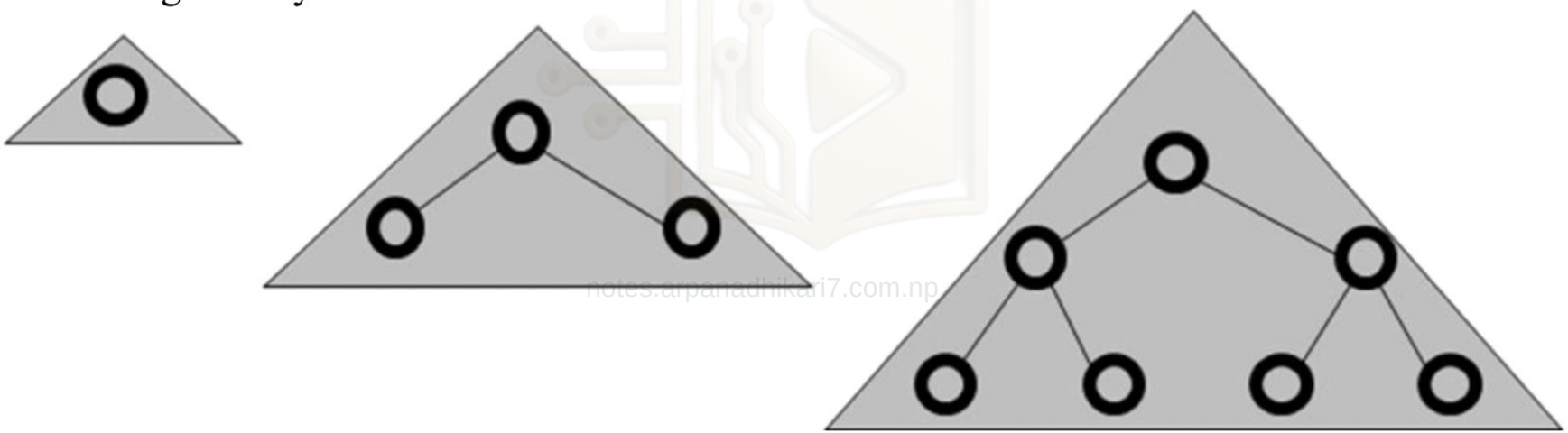
notes.arpanadhikari7.com.np

Note that depth-first search achieves its efficiency by generating the next node to explore only when this needed. The breadth-first search algorithm has to grow all the search paths available until a solution is found -and this takes up memory. Iterative deepening achieves its memory saving in the same way that depth-first search does -at the expense of redoing some computations again and again (a time cost rather than a memory one). In the search illustrated, we had to visit node d three times in all!

Available on notes.arpanadhikari7.com.np

- Complete (like BFS)
- Has linear memory requirements (like DFS)
- Classical time-space tradeoff.
- This is the preferred method for large state spaces, where the solution path length is unknown.

The overall idea goes as follows until the goal node is not found i.e. the depth limit is increased gradually.



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

## Iterative Deepening search evaluation:

Completeness:

- YES (no infinite paths)

Time complexity:

- Algorithm seems costly due to repeated generation of certain states.
- Node generation:

  level d : once
  level d-1: 2
  level d-2: 3
  …
  level 2: d-1
  level 1: d

- Total no. of nodes generated:

  $d.b + (d-1).b^2 + (d-2).b^3 + \ldots\ldots\ldots\ldots\ldots\ldots + 1.b^d = \mathbf{O(b^d)}$

Space complexity:

- It needs to store only a single path from the root node to a leaf node, along with remaining unexpanded sibling nodes for each node on the path.
- Total no. of nodes in memory:

  $1 + b + b + b + \ldots\ldots\ldots\ldots\ldots\ldots$ b d times = **O(bd)**

Available on notes.arpanadhikari7.com.np

Optimality:

- YES if path cost is non-decreasing function of the depth of the node.

*Notice that BFS generates some nodes at depth d+1, whereas IDS does not. The result is that IDS is actually faster than BFS, despite the repeated generation of node.*

**Example:** Number of nodes generated for b=10 and d=5 solution at far right

N(IDS) = 50 + 400 + 3000 + 20000 + 100000 = 123450

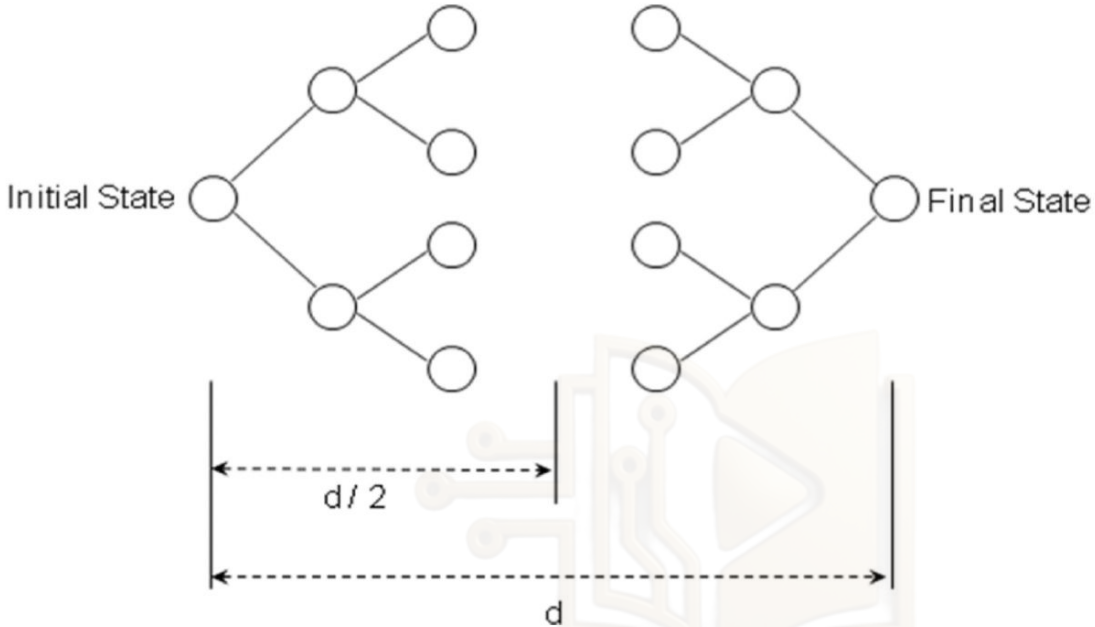
N(BFS) = 10 + 100 + 1000 + 10000 + 100000 + 999990 = 1111100

notes.arpanadhikari7.com.np

## Bidirectional Search:

This is a search algorithm which replaces a single search graph, which is likely to with two smaller graphs -- one starting from the initial state and one starting from the goal state. It then, expands nodes from the start and goal state simultaneously. Check at each stage if the nodes of one have been generated by the other, i.e, they meet in the middle. If so, the path concatenation is the solution.

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

- Completeness: yes
- Optimality: yes (If done with correct strategy- e.g. breadth first)
- Time complexity: $O(b^{d/2})$
- Space complexity: $O(b^{d/2})$

**Problems: generate predecessors; many goal states; efficient check for node already visited by other half of the search; and, what kind of search.**

Available on notes.arpanadhikari7.com.np

# Drawbacks of uninformed search

- Criterion to choose next node to expand depends only on a global criterion: level
- Does not exploit the structure of the problem
- One may prefer to use a more flexible rule, that takes advantage of what is being discovered on the way, and hunches about what can be a good move
- Very often, we can select which rule to apply by comparing the current state and the desired state.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Heuristic Search (Informed Search)

Heuristic Search Uses domain-dependent (heuristic) information in order to search the space more efficiently.

*Ways of using heuristic information:*

- Deciding which node to expand next, instead of doing the expansion in a strictly breadth-first or depth-first order;
- In the course of expanding a node, deciding which successor or successors to generate, instead of blindly generating all possible successors at one time;
- Deciding that certain nodes should be discarded, or *pruned*, from the search space.

**Heuristic Searches - Why Use?**

- It may be too resource intensive (both time and space) to use a blind search
- Even if a blind search will work we may want a more efficient search method

notes.arpanadhikari7.com.np

Informed Search uses domain specific information to improve the search pattern

- Define a heuristic function, h(n), that estimates the "goodness" of a node n.
- Specifically, h(n) = estimated cost (or distance) of minimal cost path from n to a goal state.
- The heuristic function is an estimate, based on domain-specific information that is computable from the current state description, of how close we are to a goal.

Available on notes.arpanadhikari7.com.np

## Best-First Search

**Idea:** use an *evaluation function f(n)* that gives an indication of which node to expand next for each node.

- usually gives an estimate to the goal.
- the node with the lowest value is expanded first.

A key component of *f(n)* is a heuristic function, *h(n),* which is a additional knowledge of the problem.

There is a whole family of best-first search strategies, each with a different evaluation function.

Typically, strategies use estimates of the cost of reaching the goal and try to minimize it.

Special cases: based on the evaluation function.

- Greedy best-first search
- A*search

Available on notes.arpanadhikari7.com.np

## Greedy Best First Search

The best-first search part of the name means that it uses an evaluation function to select which node is to be expanded next. The node with the lowest evaluation is selected for expansion because that is the *best* node, since it supposedly has the closest path to the goal (if the heuristic is good). Unlike A* which uses both the link costs and a heuristic of the cost to the goal, greedy best-first search uses only the heuristic, and not any link costs. A disadvantage of this approach is that if the heuristic is not accurate, it can go down paths with high link cost since there might be a low heuristic for the connecting node.

Evaluation function $f(n) = h(n)$ (heuristic) = estimate of cost from $n$ to *goal*.

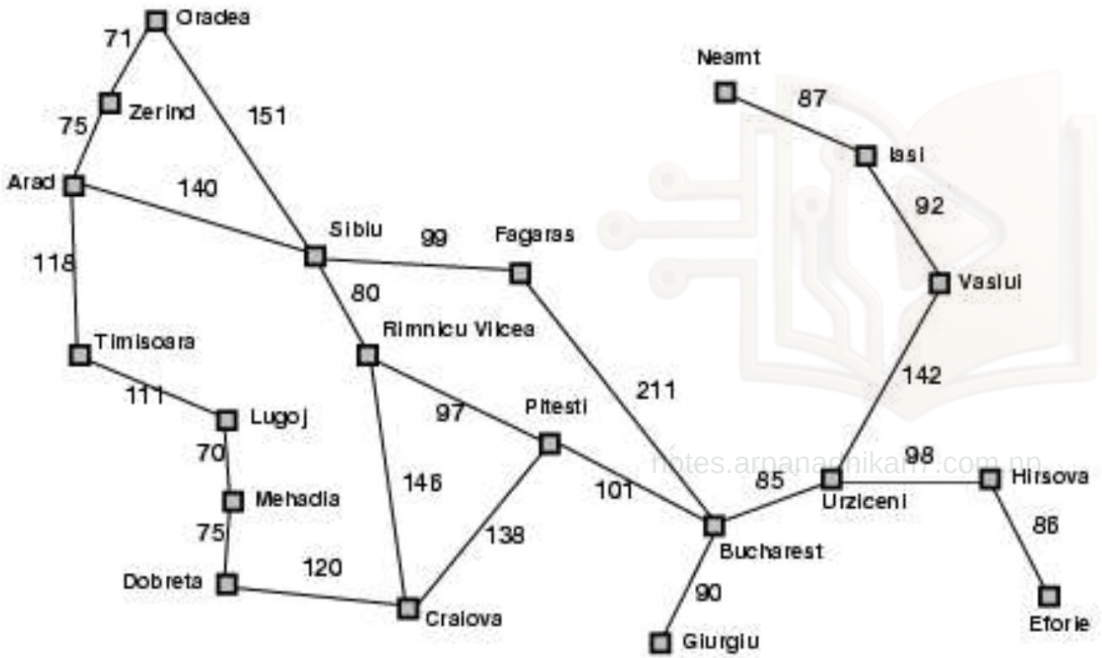
e.g., $h_{SLD}(n)$ = straight-line distance from $n$ to goal

notes.arpanadhikari7.com.np

Greedy best-first search expands the node that appears to be closest to goal. The greedy best-first search algorithm is $O(b^m)$ in terms of space and time complexity. (Where $b$ is the average branching factor (the average number of successors from a state), and $m$ is the maximum depth of the search tree.)

Available on notes.arpanadhikari7.com.np

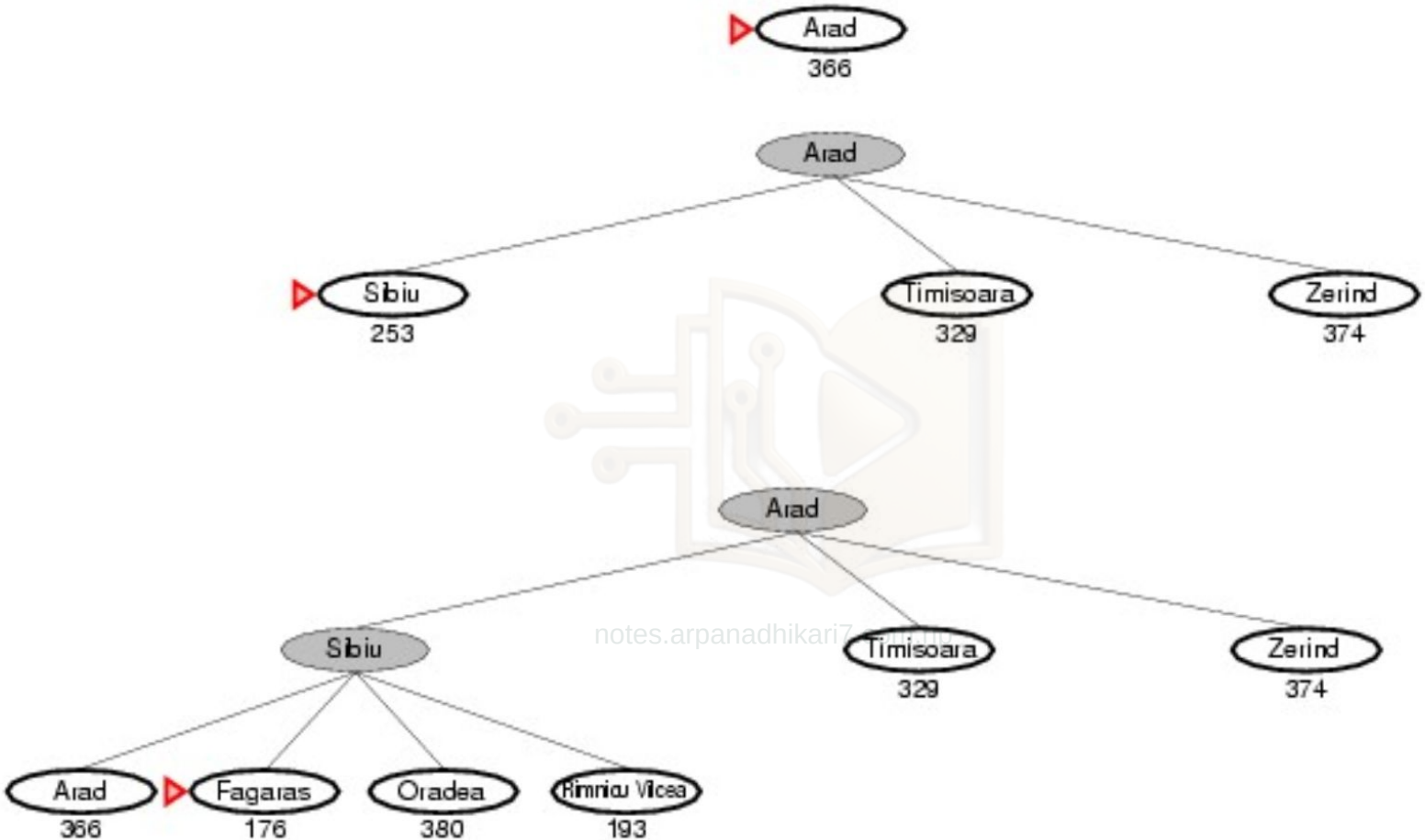
**Example: Given following graph of cities, starting at Arad city, problem is to reach to the Bucharest.**




| Straight-line distance to Bucharest | |
|---|---|
| Arad | 366 |
| Bucharest | 0 |
| Craiova | 160 |
| Dobreta | 242 |
| Eforie | 161 |
| Fagaras | 176 |
| Giurgiu | 77 |
| Hirsova | 151 |
| Iasi | 226 |
| Lugoj | 244 |
| Mehadia | 241 |
| Neamt | 234 |
| Oradea | 380 |
| Pitesti | 10 |
| Rimnicu Vilcea | 193 |
| Sibiu | 253 |
| Timisoara | 329 |
| Urziceni | 80 |
| Vaslui | 199 |
| Zerind | 374 |

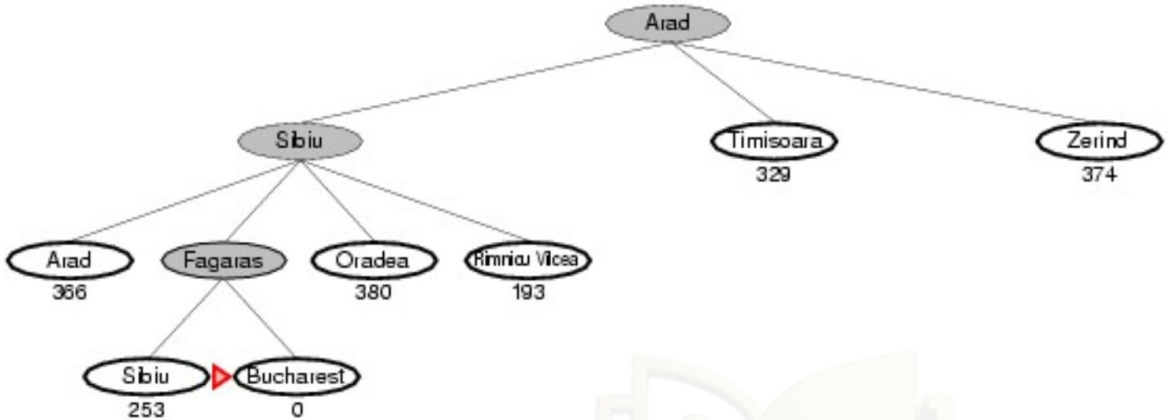
**Solution using greedy best first can be as below:**

Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




**Greedy Best-first search**

- minimizes estimated cost h(n) from current node n to goal;
- is informed but (almost always) suboptimal and incomplete.

**Admissible Heuristic:**

A heuristic function is said to be **admissible** if it is no more than the lowest-cost path to the goal. In other words, a heuristic is admissible if it never overestimates the cost of reaching the goal. An admissible heuristic is also known as an **optimistic heuristic**.

An admissible heuristic is used to estimate the cost of reaching the goal state in an informed search algorithm. In order for a heuristic to be admissible to the search problem, the estimated cost must always be lower than the actual cost of reaching the goal state. The search algorithm uses the admissible heuristic to find an estimated optimal path to the goal

Available on notes.arpanadhikari7.com.np

state from the current node. For example, in A* search the evaluation function (where $n$ is the current node) is: $f(n) = g(n) + h(n)$

where;

$f(n)$ = the evaluation function.
$g(n)$ = the cost from the start node to the current node
$h(n)$ = estimated cost from current node to goal.

$h(n)$ is calculated using the heuristic function. *With a non-admissible heuristic, the A* algorithm would overlook the optimal solution to a search problem due to an overestimation in f(n).*

notes.arpanadhikari7.com.np

*It is obvious that the SLD heuristic function is admissible as we can never find a shorter distance between any two towns.*

Available on notes.arpanadhikari7.com.np

## A* Search:

A* is a best first, informed graph search algorithm. A* is different from other best first search algorithms in that it uses a heuristic function h(x) as well as the path cost to the node g(x), in computing the cost f(x) = h(x) + g(x) for the node. The $h(x)$ part of the $f(x)$ function must be an admissible heuristic; that is, it must not overestimate the distance to the goal. Thus, for an application like routing, $h(x)$ might represent the straight-line distance to the goal, since that is physically the smallest possible distance between any two points or nodes.

**It finds a minimal cost-path joining the start node and a goal node for node n.**
**Evaluation function: f(n) = g(n) + h(n)**

Where,

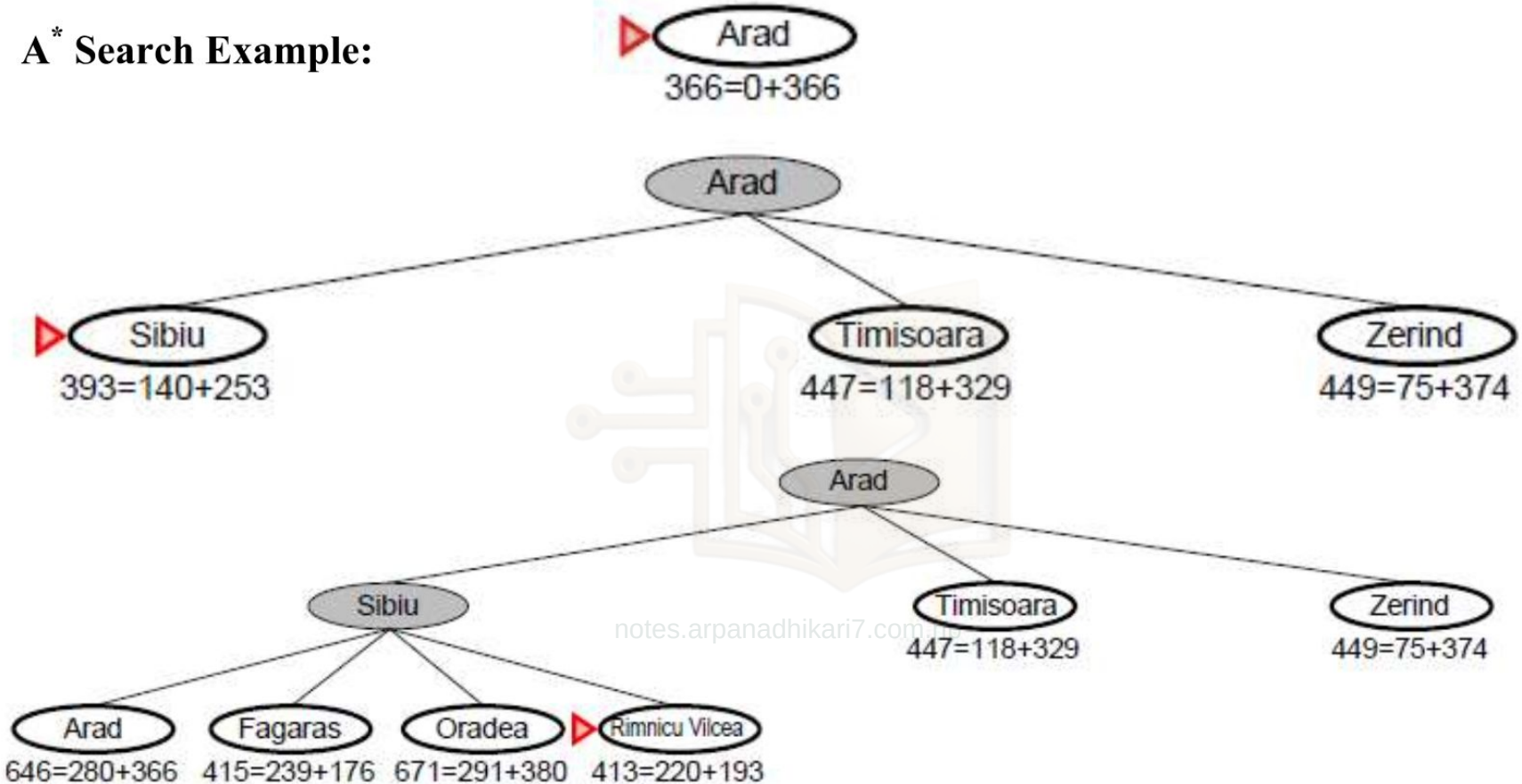
g(n) = cost so far to reach n from root
h(n) = estimated cost to goal from n
f(n) = estimated total cost of path through n to goal

- combines the two by minimizing f(n) = g(n) + h(n);
- is informed and, *under reasonable assumptions*, optimal and complete.

As A* traverses the graph, it follows a path of the lowest *known* path, keeping a sorted priority queue of alternate path segments along the way. If, at any point, a segment of the path being traversed has a higher cost than another encountered path segment, it abandons the higher-cost path segment and traverses the lower-cost path segment instead. This process continues until the goal is reached.

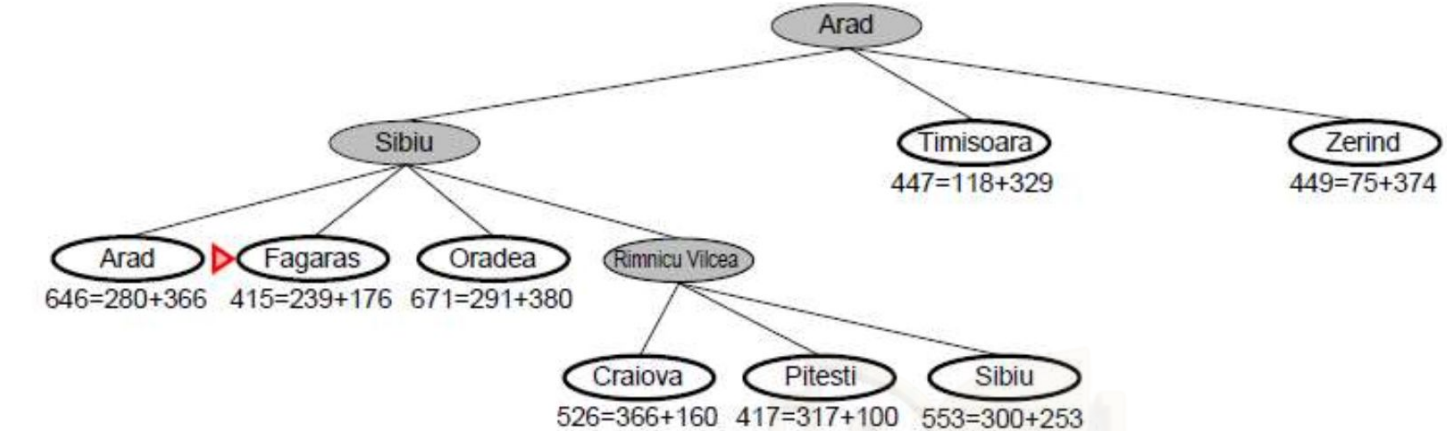
Available on notes.arpanadhikari7.com.np

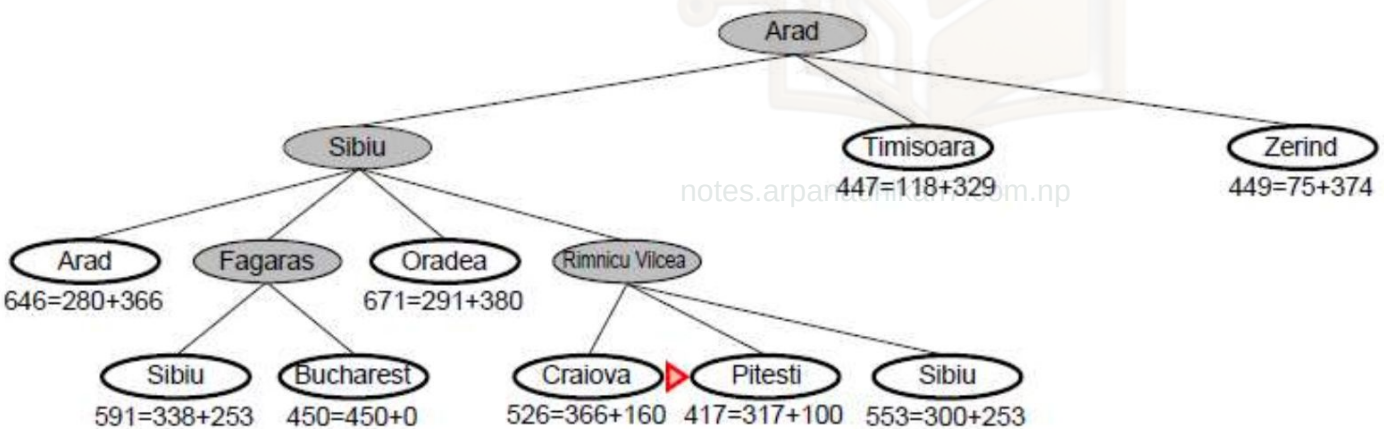
## A* Search Example:




notes.arpanadhikari7.com.np

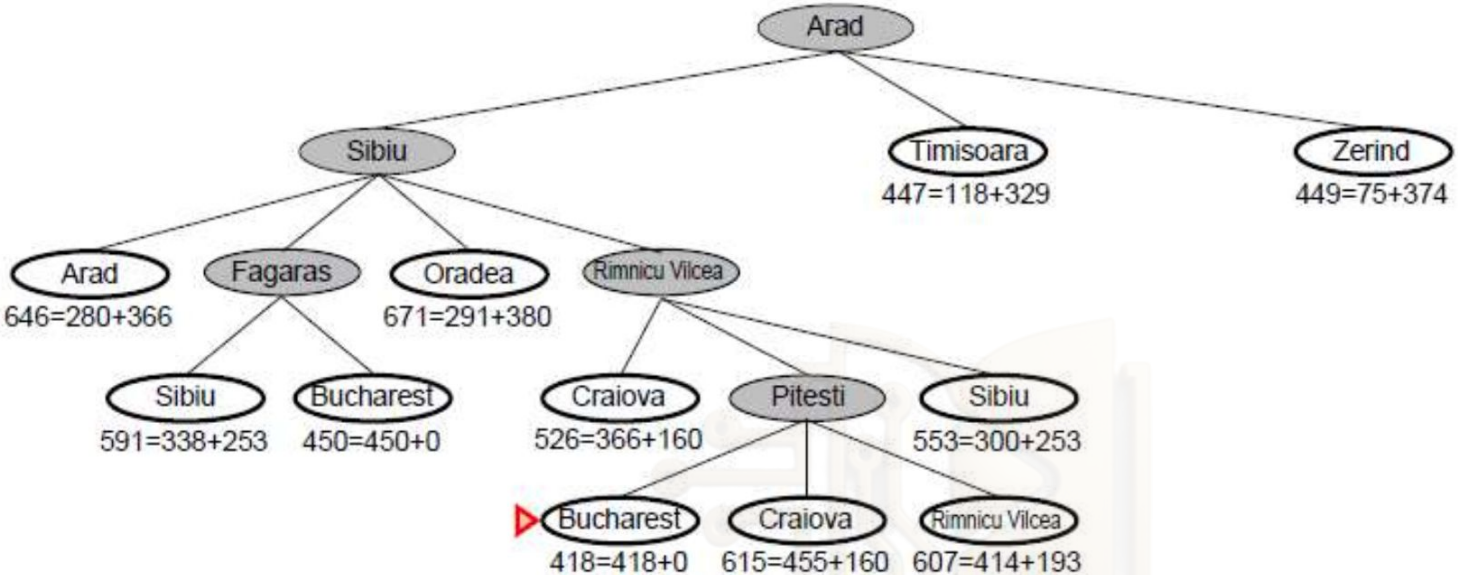
Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




**Admissibility and Optimality:**

notes.arpanadhikari7.com.np

A* is admissible and considers fewer nodes than any other admissible search algorithm with the same heuristic. This is because A* uses an "optimistic" estimate of the cost of a path through every node that it considers—optimistic in that the true cost of a path through that node to the goal will be at least as great as the estimate. But, critically, as far as A* "knows", that optimistic estimate might be achievable.

Available on notes.arpanadhikari7.com.np

# Hill Climbing Algorithm

- A search method of selecting the best local choice at each step in hopes of finding an optimal solution.
- Does not consider how optimal the current solution is.
- Does not consider steps past the next immediate choice.
- Steps:
  1. Define an evaluation function f(x) to determine the value of a state.
  2. From the current state, determine the search space (actions) for one step ahead.
  3. Select the action from the search space that returns the highest value.

notes.arpanadhikari7.com.np

# Hill Climbing

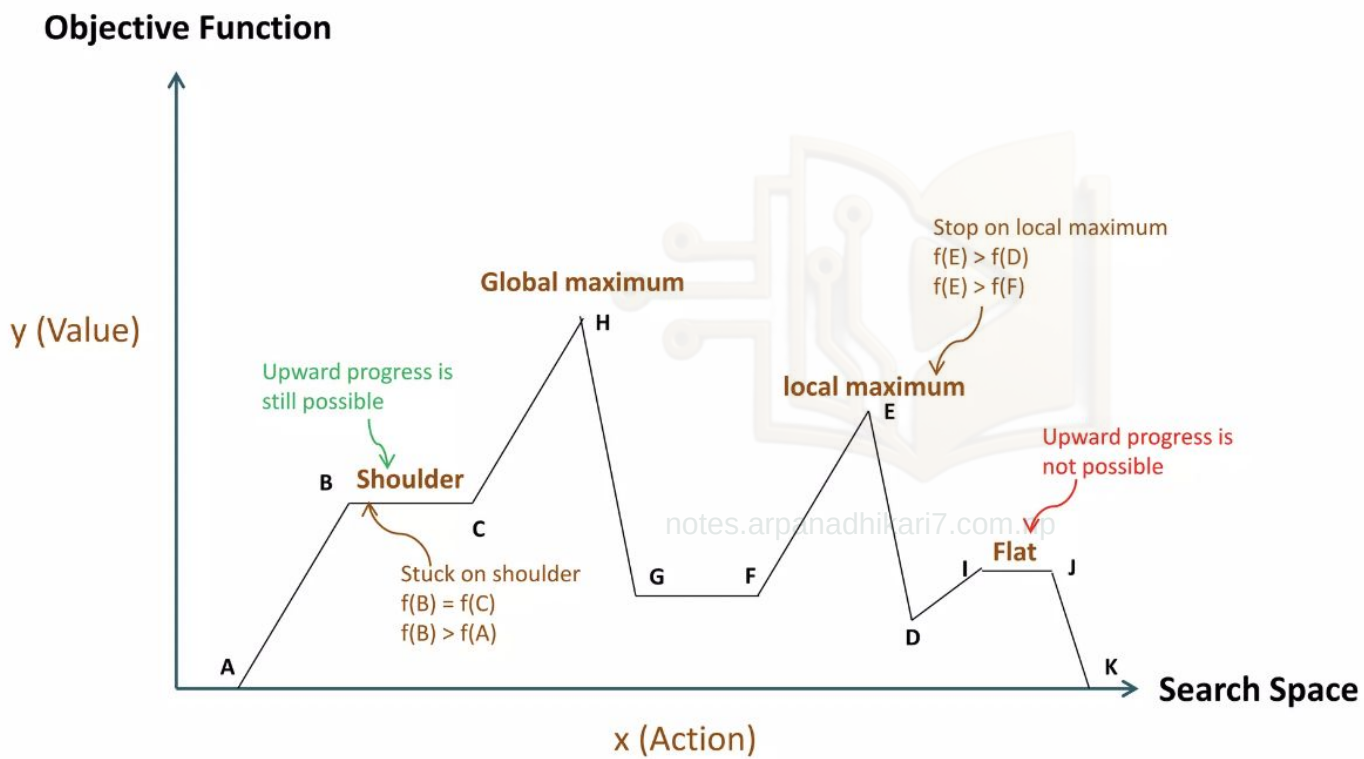
Hill climbing can be used to solve problems that have many solutions, some of which are better than others.

It starts with a random (potentially poor) solution, and iteratively makes small changes to the solution, each time improving it a little.

When the algorithm cannot see any improvement anymore, it terminates. Ideally, at that point the current solution is close to optimal, but it is not guaranteed that hill climbing will ever come close to the optimal solution.

notes.arpanadhikari7.com.np

# Hill Climbing – Objective Function




notes.arpanadhikari7.com.np

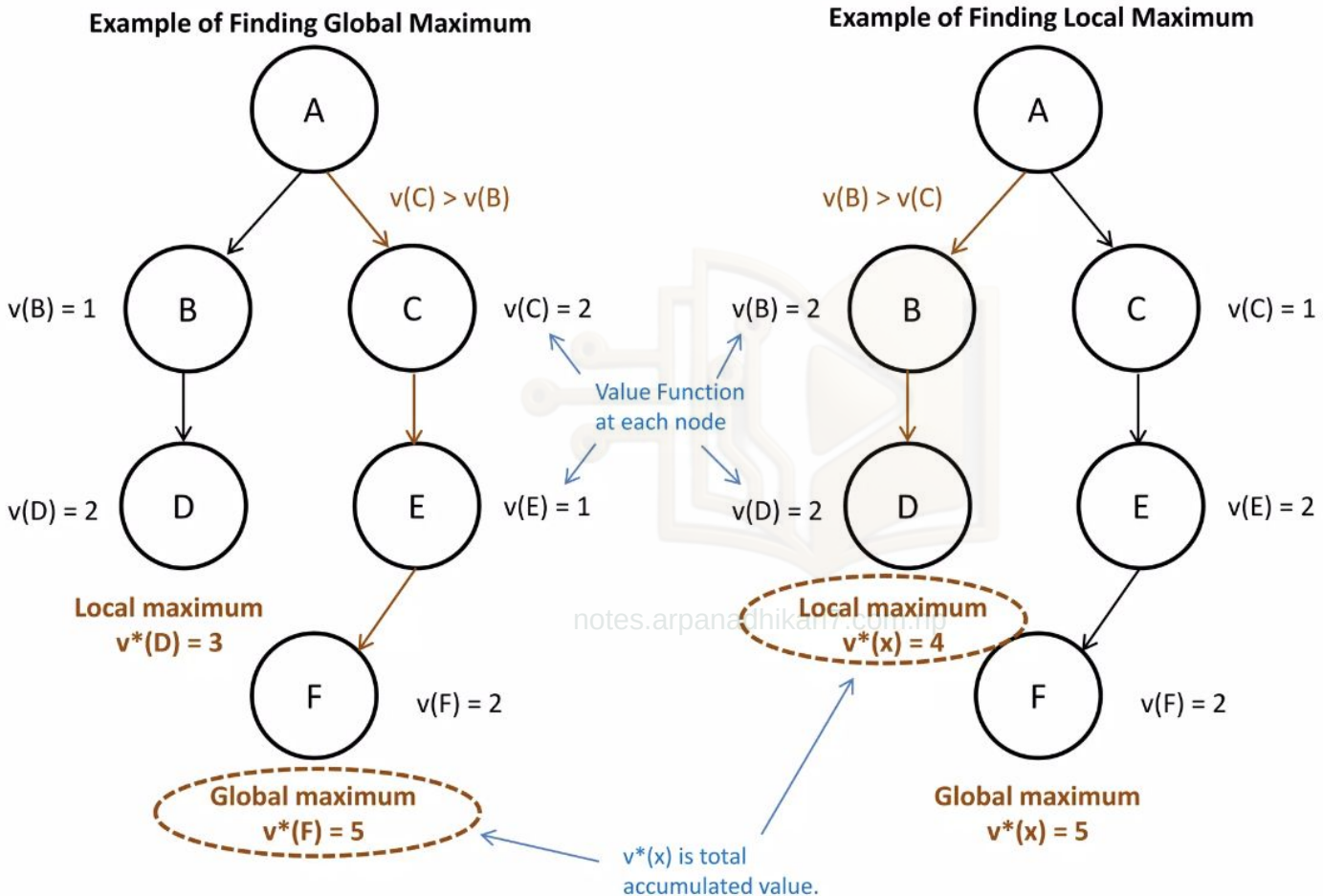
The hill climbing can be described as follows:

- Start with *current-state* = initial-state.
- Until *current-state* = goal-state OR there is no change in *current-state* do:
  - Get the successors of the current state and use the evaluation function to assign a score to each successor.
  - If one of the successors has a better score than the current-state then set the new current-state to be the successor with the best score.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Value Function




Available on notes.arpanadhikari7.com.np

# Algorithm

```
function HillClimb( initial )

        initialize node with initial                    # set the current node to the starting point

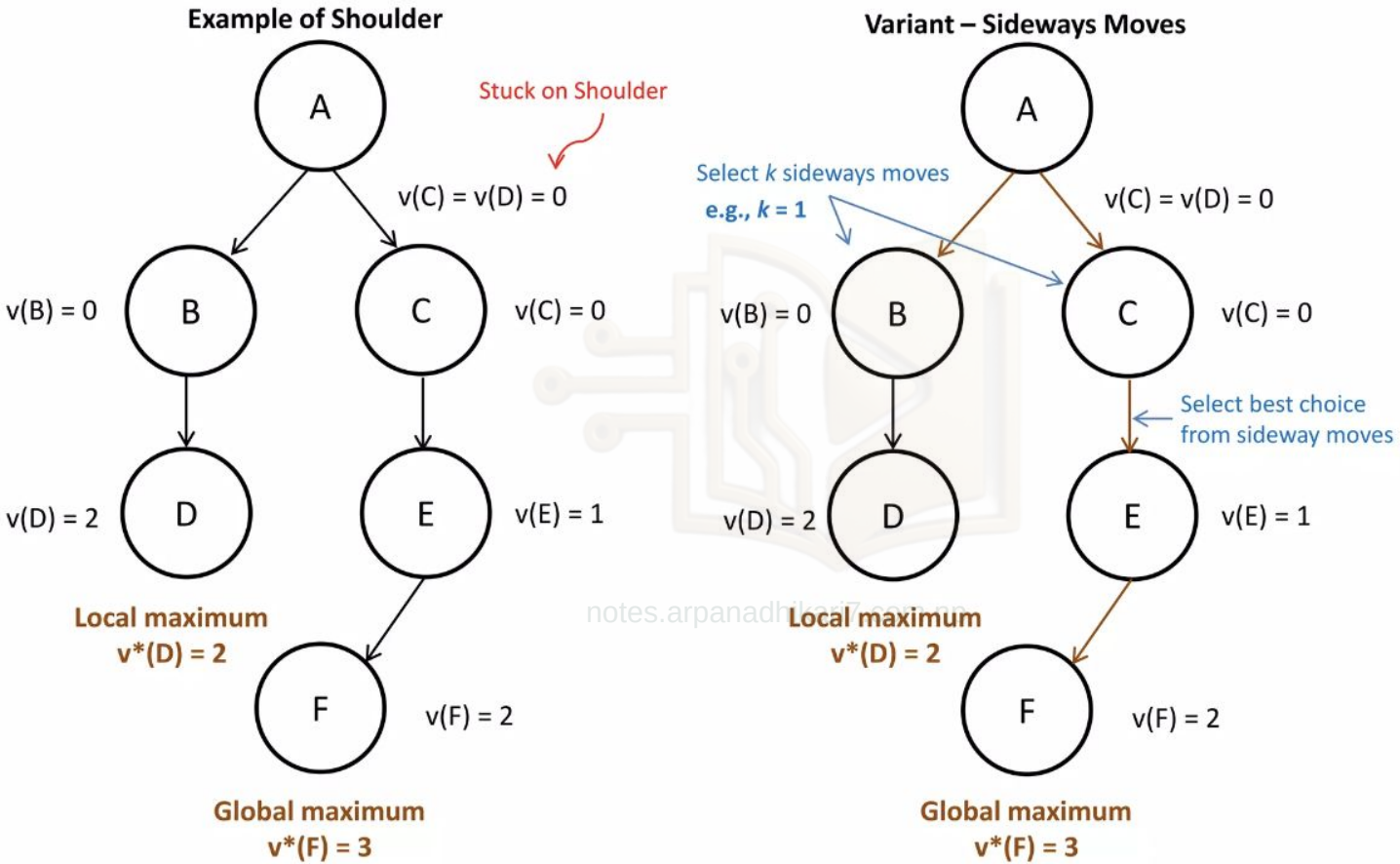
        while forever                                   # continue until you cannot climb higher
                initialize max  to minus infinity       # minimum value
                for each child (neighbor) of the node
                        if v(child) >  max              # find neighbor with max value
                                max = v(child)
                                next = child
                if max <= 0                             # cannot climb higher
                        return node

                node = next                             # climb to the next node
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Sideways Moves – Escape Shoulder




Available on notes.arpanadhikari7.com.np

# Algorithm – Sideways Moves

```
function HillClimb( initial, k )        ← Add parameter for number of sideways moves

    initialize node with initial                    # set the current node to the starting point

    while forever                                   # continue until you cannot climb higher
        initialize max  to minus infinity           # minimum value
        for each child (neighbor) of the node
            if v(child) >  max                      # find neighbor with max value
                max = v(child)
                next = child
        if max <= 0                                 # cannot climb higher
No sideways moves ←     if k == 0
                        return node
                    for each child (neighbor) of the node
                        value = HillClimb( child, k-1)   ← Decrease k by one
                        if value > max
                            max = value
                            next = child
                    if max <= 0
                        return node
        node = next                                 # climb to the next node
```

Recursively call the HillClimb algorithm on each child, while decrementing the number of sideways moves (depth)

Available on notes.arpanadhikari7.com.np

# Variant - Stochastic

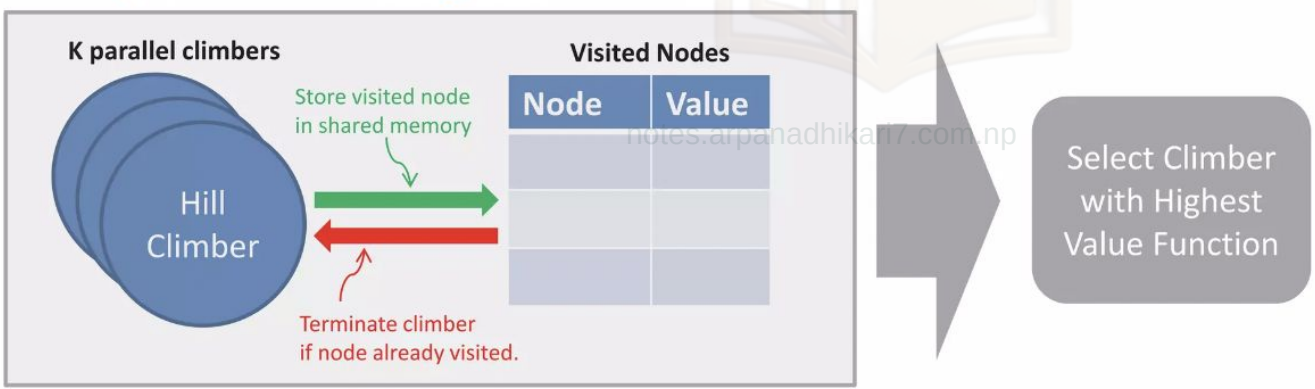
- **Stochastic – choose node at random**
  - **If none of the child (neighbor) nodes has a value > 0, then choose one of the nodes at random.**
  - **There needs to be a cutoff (k) of random choices, or the method will go into an infinite loop (e.g., you are at the global maximum).**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Variant - Local Beam Search

- Search with multiple (k) Hill Climbers in parallel.
  - Select k different initial states (nodes).
  - Communicate between climbers which states (nodes) have been visited.
  - Terminate Hill Climber if state (node) already visited by another Hill Climber – eliminates duplication in search.
  - When all Hill Climbers reach terminal state (node), select path with the highest value function.




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Problems with Hill Climbing

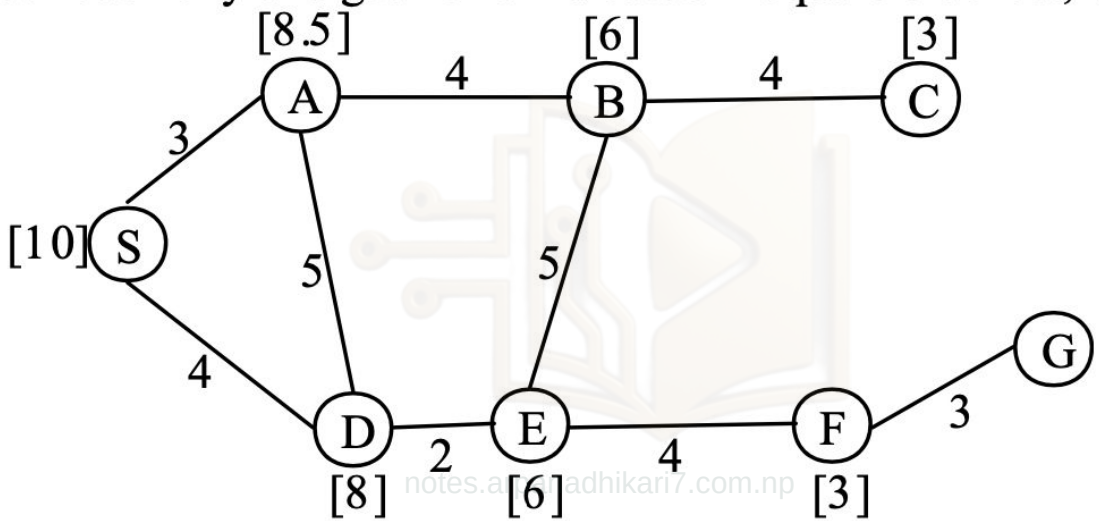
Gets stuck at local minima when we reach a position where there are no better neighbors, it is not a guarantee that we have found the best solution. Ridge is a sequence of local maxima.

Another type of problem we may find with hill climbing searches is finding a ***plateau.*** This is an area where the search space is flat so that all neighbors return the same evaluation

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

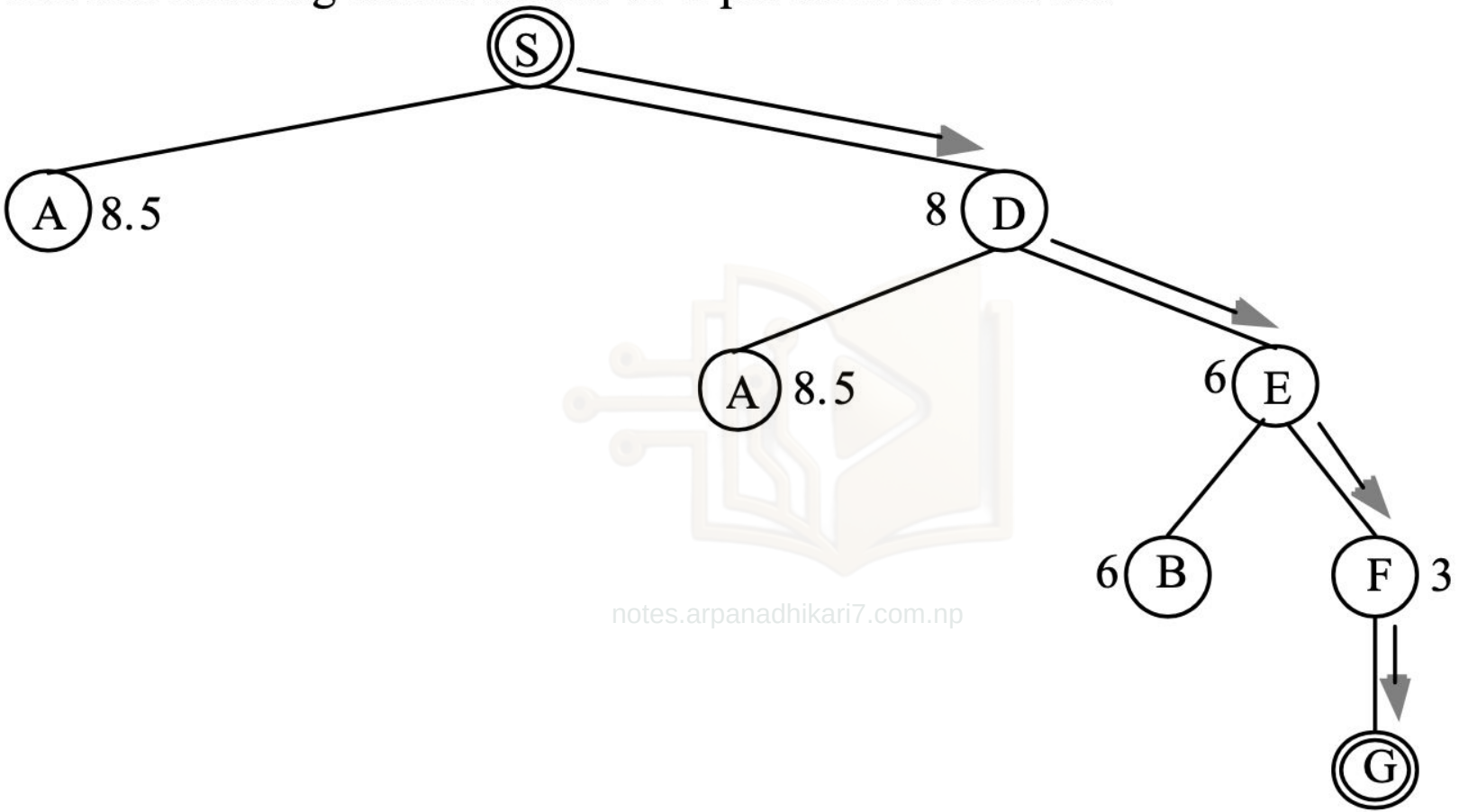
For instance, consider that the most promising successor of a node is the one that has the shortest straight-line distance to the goal node G. In figure below, the straight line distances between each city and goal G is indicated in square brackets, i.e. the heuristic.




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

The hill climbing search from S to G proceeds as follows:

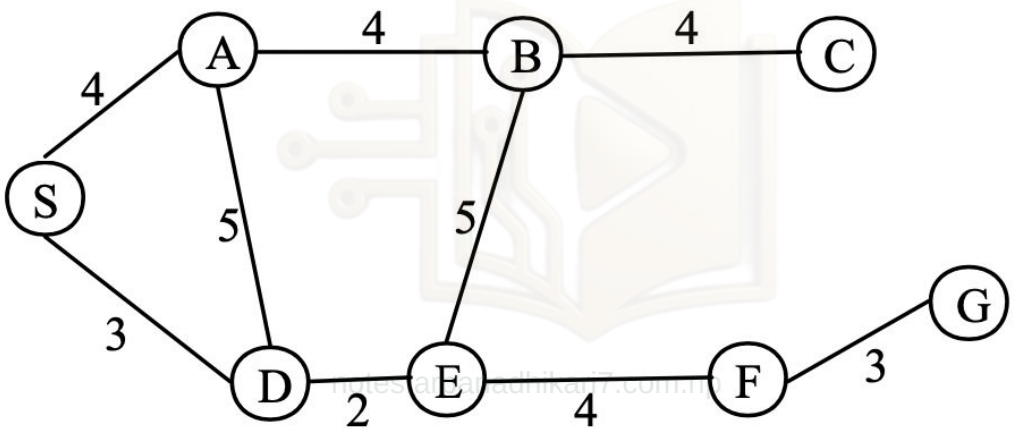



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

**Exercise:**

Apply the hill climbing algorithm to find a path from S to G, considering that the most promising successor of a node is its closest neighbor.




Available on notes.arpanadhikari7.com.np

# Simulated Annealing

It is motivated by the physical annealing process in which material is heated and slowly cooled into a uniform structure.

Compared to hill climbing the main difference is that SA allows downwards steps.

Simulated annealing also differs from hill climbing in that a move is selected at random and then decides whether to accept it.

notes.arpanadhikari7.com.np

If the move is better than its current position then simulated annealing will always take it.

If the move is worse (i.e. lesser quality) then it will be accepted based on some probability.

Available on notes.arpanadhikari7.com.np

The probability of accepting a worse state is given by the equation

$$P = \text{exponential}(-c / t) > r$$

Where

| | | |
|---|---|---|
| $c$ | = | the change in the evaluation function |
| $t$ | = | the current value |
| $r$ | = | a random number between 0 and 1 |

The probability of accepting a worse state is a function of both the current value and the change in the cost function. The most common way of implementing an SA algorithm is to implement hill climbing with an accept function and modify it for SA

Available on notes.arpanadhikari7.com.np

# Puzzle problem

**N-Puzzle** or **sliding puzzle** is a popular puzzle that consists of N tiles where N can be 8, 15, 24, and so on. In our example N = 8.

The puzzle is divided into sqrt(N+1) rows and sqrt(N+1) columns. Eg. 15-Puzzle will have 4 rows and 4 columns and an 8-Puzzle will have 3 rows and 3 columns.

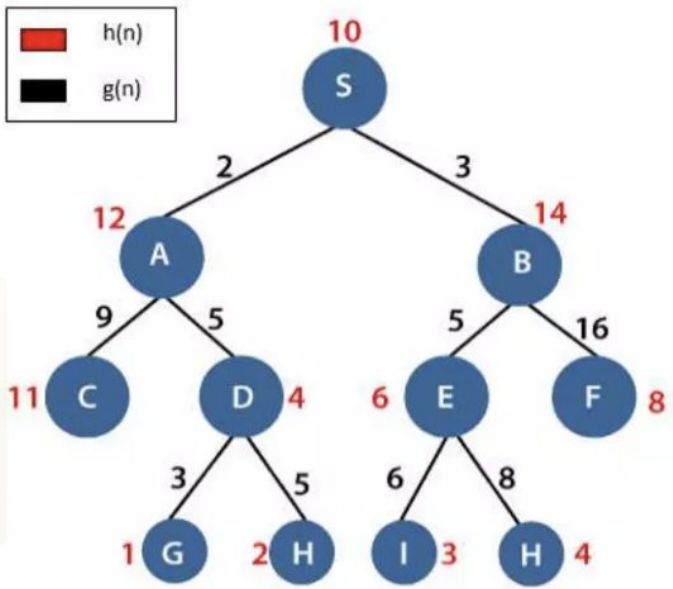
The puzzle consists of N tiles and one empty space where the tiles can be moved. Start and Goal configurations (also called state) of the puzzle are provided.

The puzzle can be solved by moving the tiles one by one in the single empty space and thus achieving the Goal configuration.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

A Heuristic is a technique to solve a problem faster than classic methods, or to find an approximate solution when classic methods cannot. This is a kind of a shortcut as we often trade one of optimality, completeness, accuracy, or precision for speed.



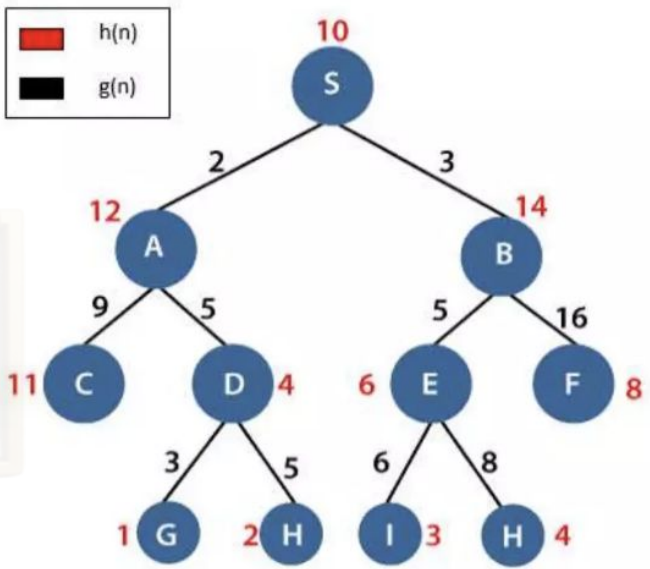
A*Search

notes.arpanadhikari7.com.np

1. A* Algorithm maintains a tree of paths originating at the initial state.

2. It extends those paths one edge at a time.

3. It continues until final state is reached.

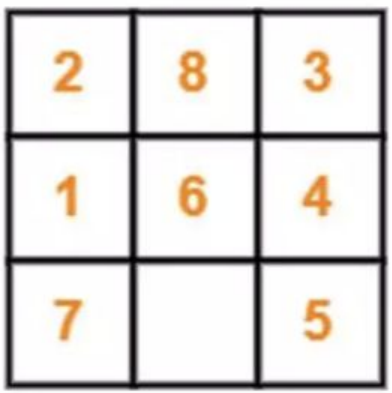


A*Search

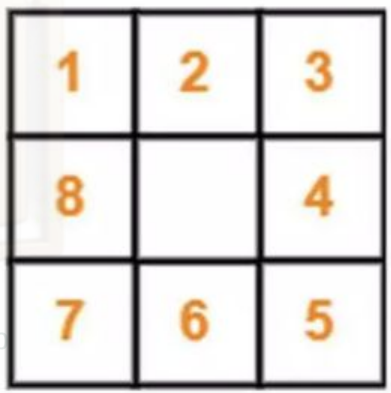
notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

**Given an initial state of a 8-puzzle problem and final state to be reached-**




Initial State




Final State

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

| | | |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| 7 | | 5 |

Initial State

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 8 | | 4 |
| 7 | 6 | 5 |

Final State

**Initial State**

| | | |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| 7 | | 5 |

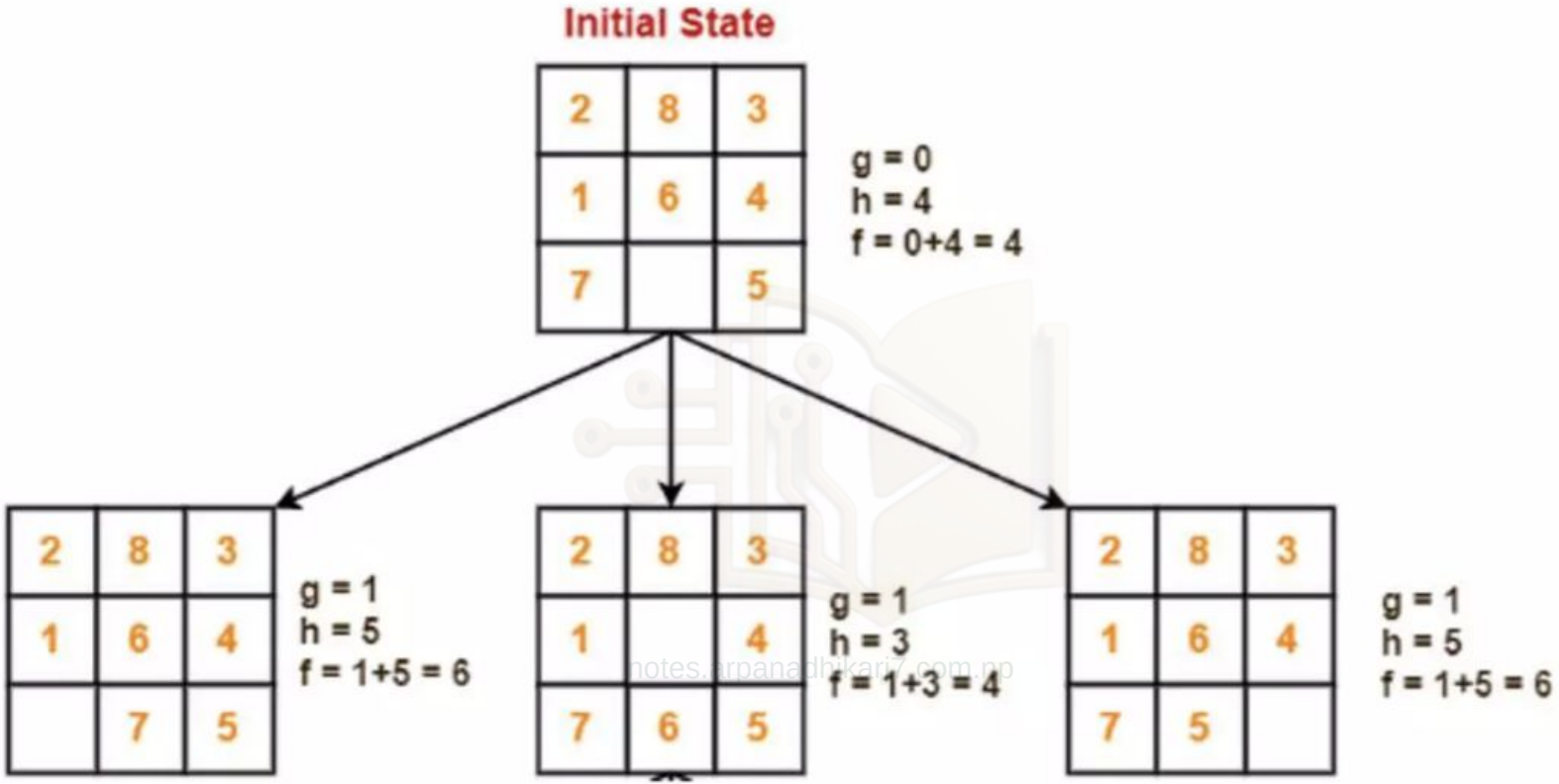
g = 0
h = 4
f = 0+4 = 4

**Consider ,**
**g(n) = Depth of node**
**and**
**h(n) = Number of misplaced tiles.**

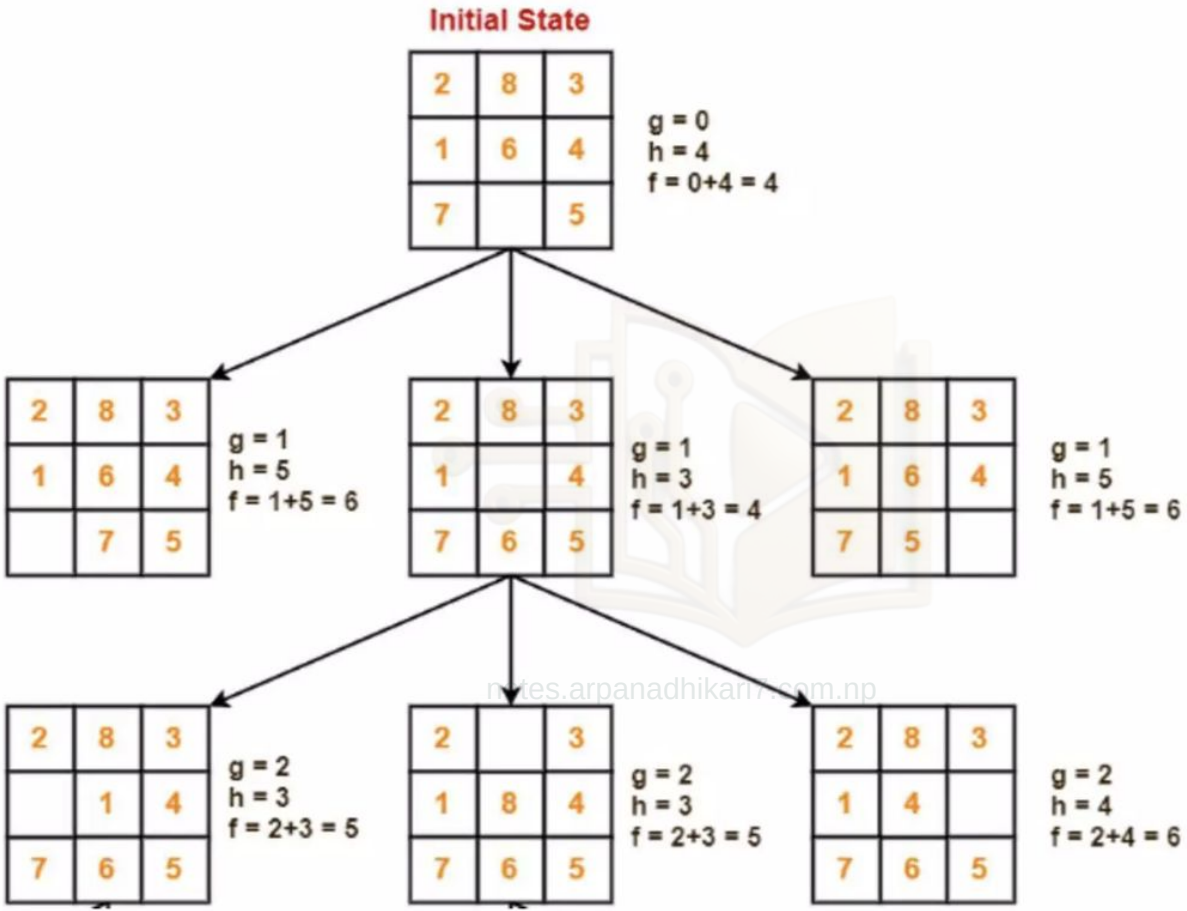
1. Up
2. Down
3. Right or
4. Left

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

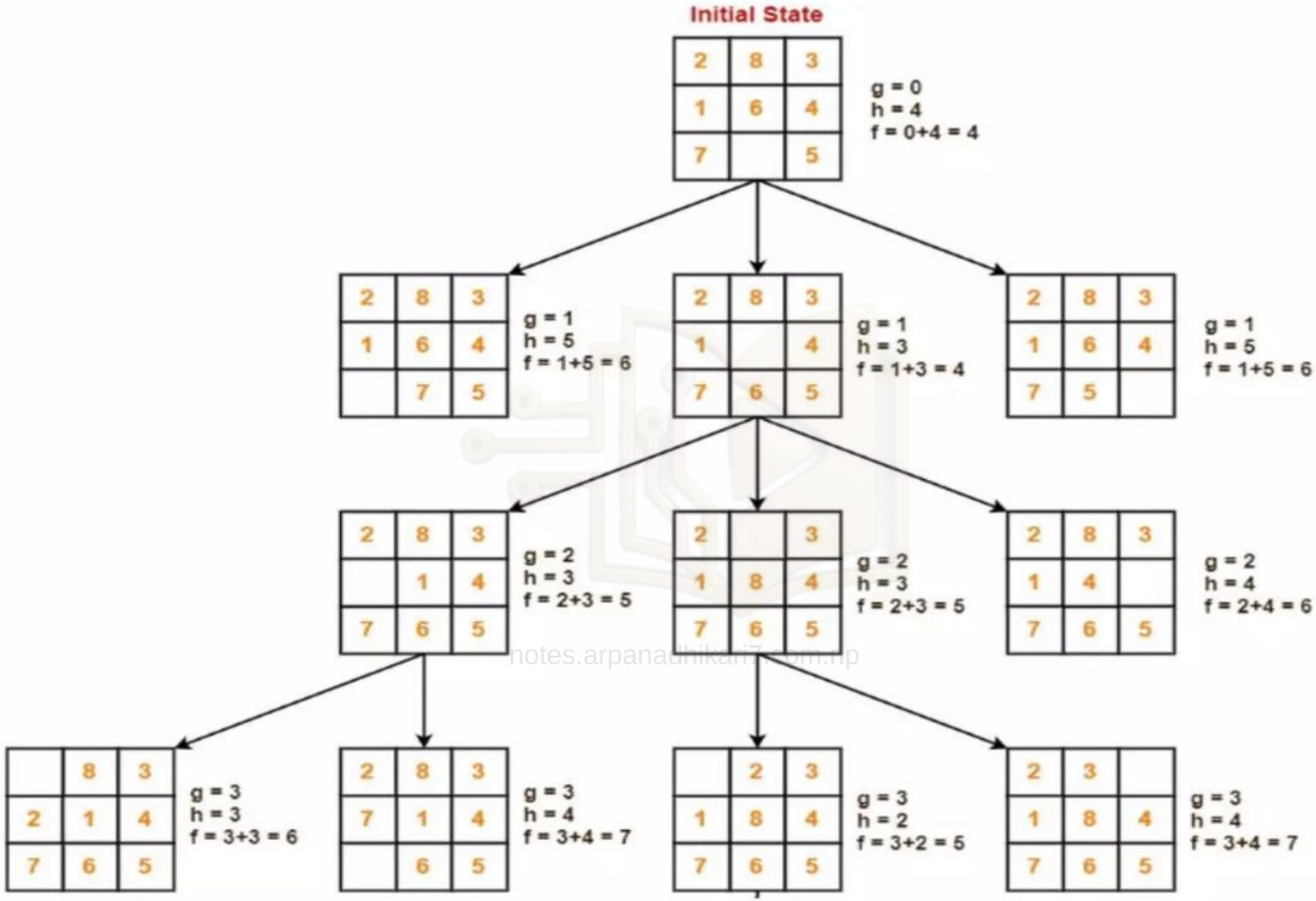



Available on notes.arpanadhikari7.com.np




notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np




Available on notes.arpanadhikari7.com.np

## Pros:

- It is complete and optimal.
- It is the best one from other techniques. It is used to solve very complex problems.
- It is optimally efficient, i.e. there is no other optimal algorithm guaranteed to expand fewer nodes than A*.

## Cons:

- This algorithm is complete if the branching factor is finite and every action has fixed cost.
- The speed execution of A* search is highly dependent on the accuracy of the heuristic algorithm that is used to compute h (n).
- It has complexity problems.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np










notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# Game playing

Game Playing is an important domain of artificial intelligence. Games don't require much knowledge; the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game.

Both players try to win the game. So, both of them try to make the best move possible at each turn. Searching techniques like BFS(Breadth First Search) are not accurate for this as the branching factor is very high, so searching will take a lot of time. So, we need another search procedures that improve –

- **Generate procedure** so that only good moves are generated.
- **Test procedure** so that the best move can be explored first.

notes.arpanadhikari7.com.np

The most common search technique in game playing is Minimax search procedure. It is depth-first depth-limited search procedure. It is used for games like chess and tic-tac-toe.

Available on notes.arpanadhikari7.com.np

A game can be formally defined as a kind of search problem as below:

Initial state: It includes the board position and identifies the player to move.

Successor function: It gives a list of (move, state) pairs each indicating a legal move and resulting state.

Terminal test: This determines when the game is over. States where the game is ended are called terminal states.

Utility function: It gives numerical value of terminal states. E.g. win (+1), loose (-1) and draw (0). Some games have a wider variety of possible outcomes eg. ranging from +92 to -192.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

# MinMax Algorithm

Let us assign the following values for the game: 1 for win by X, 0 for draw, -1 for loss by X.

Given the values of the terminal nodes (win for X (1), loss for X (-1), or draw (0)), the values of the non-terminal nodes are computed as follows:

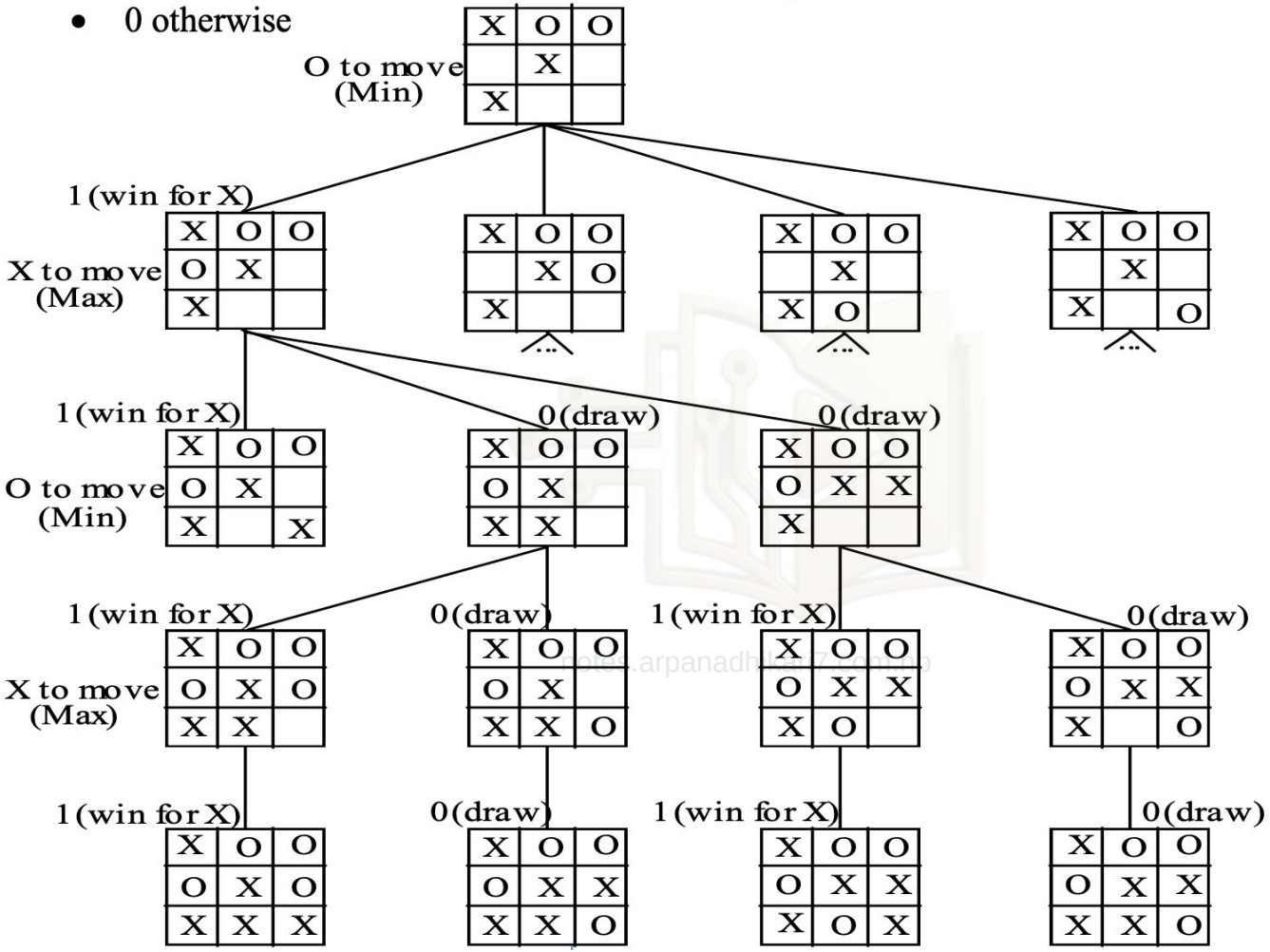
1. the value of a node where it is the turn of player X to move is the maximum of the values of its successors (because X tries to maximize its outcome);

2. the value of a node where it is the turn of player O to move is the minimum of the values of its successors (because O tries to minimize the outcome of X).

Figure below shows how the values of the nodes of the search tree are computed from the values of the leaves of the tree. The values of the leaves of the tree are given by the rules of

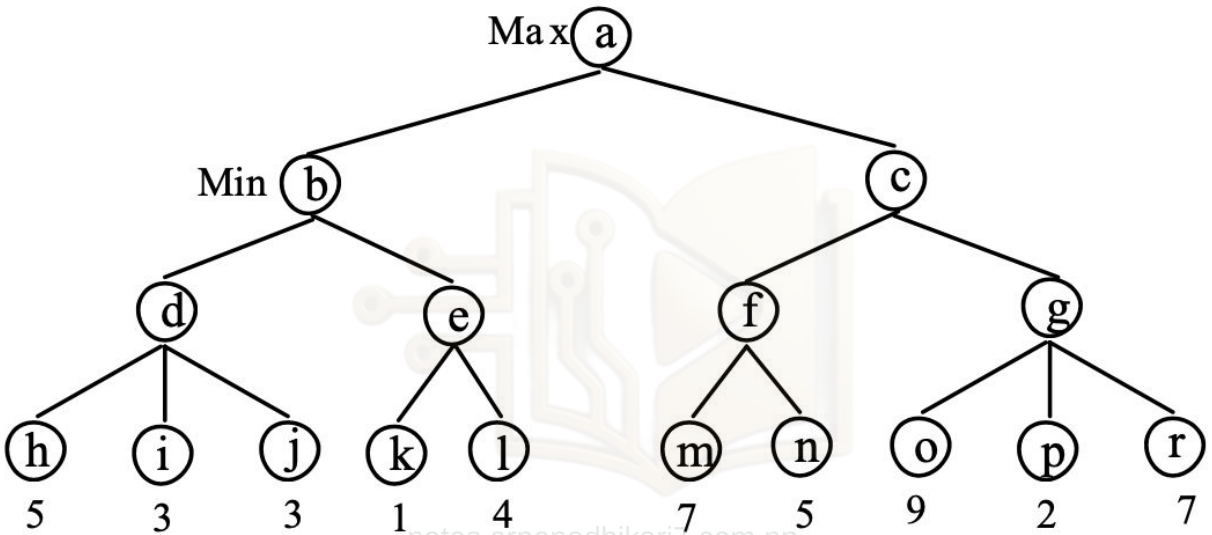
notes.arpanadhikari7.com.np

- 1 if there are three X in a row, column or diagonal;
- -1 if there are three O in a row, column or diagonal;
- 0 otherwise




Available on notes.arpanadhikari7.com.np

Consider the following game tree (drawn from the point of view of the Maximizing player):




notes.arpanadhikari7.com.np

Show what moves should be chosen by the two players, assuming that both are using the mini-max procedure.

Available on notes.arpanadhikari7.com.np

# Alpha beta pruning

The problem with minimax search is that the number if game states it has examine is exponential in the number of moves. Unfortunately, we can't eliminate the exponent, but we can effectively cut it in half.

The idea is to compute the correct minimax decision without looking at every node in the game tree, which is the concept behind pruning.

Here idea is to eliminate large parts of the tree from consideration.

The particular technique for pruning that we will discuss here is —Alpha-Beta Pruning‖.

When this approach is applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.

Alpha-beta pruning can be applied to trees of any depth, and it is often possible to prune entire sub-trees rather than just leaves.

Available on notes.arpanadhikari7.com.np

Alpha-beta pruning is a technique for evaluating nodes of a game tree that eliminates unnecessary evaluations. It uses two parameters, alpha and beta.

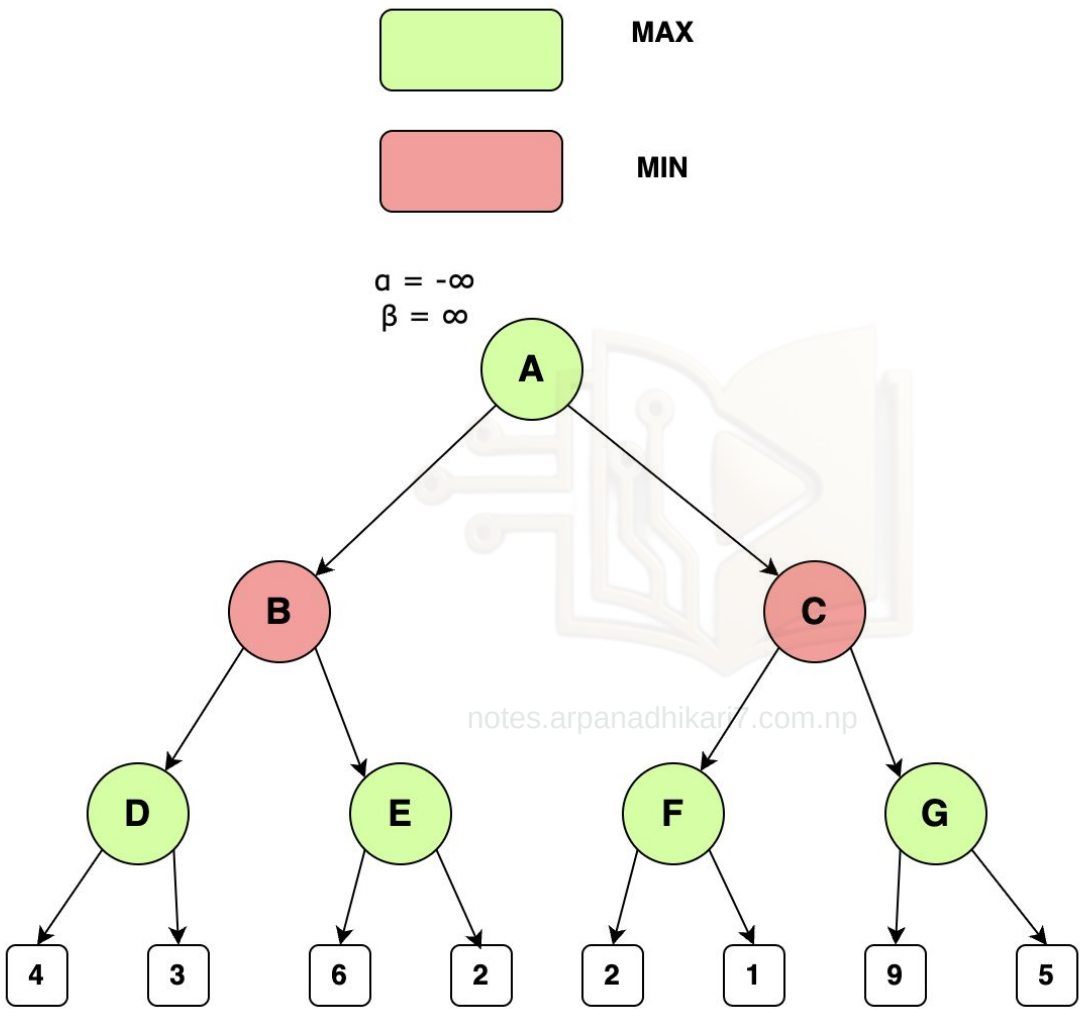
Alpha: is the value of the best (i.e. highest value) choice we have found so far at any choice point along the path for MAX.

Beta: is the value of the best (i.e. lowest-value) choice we have found so far at any choice point along the path for MIN.

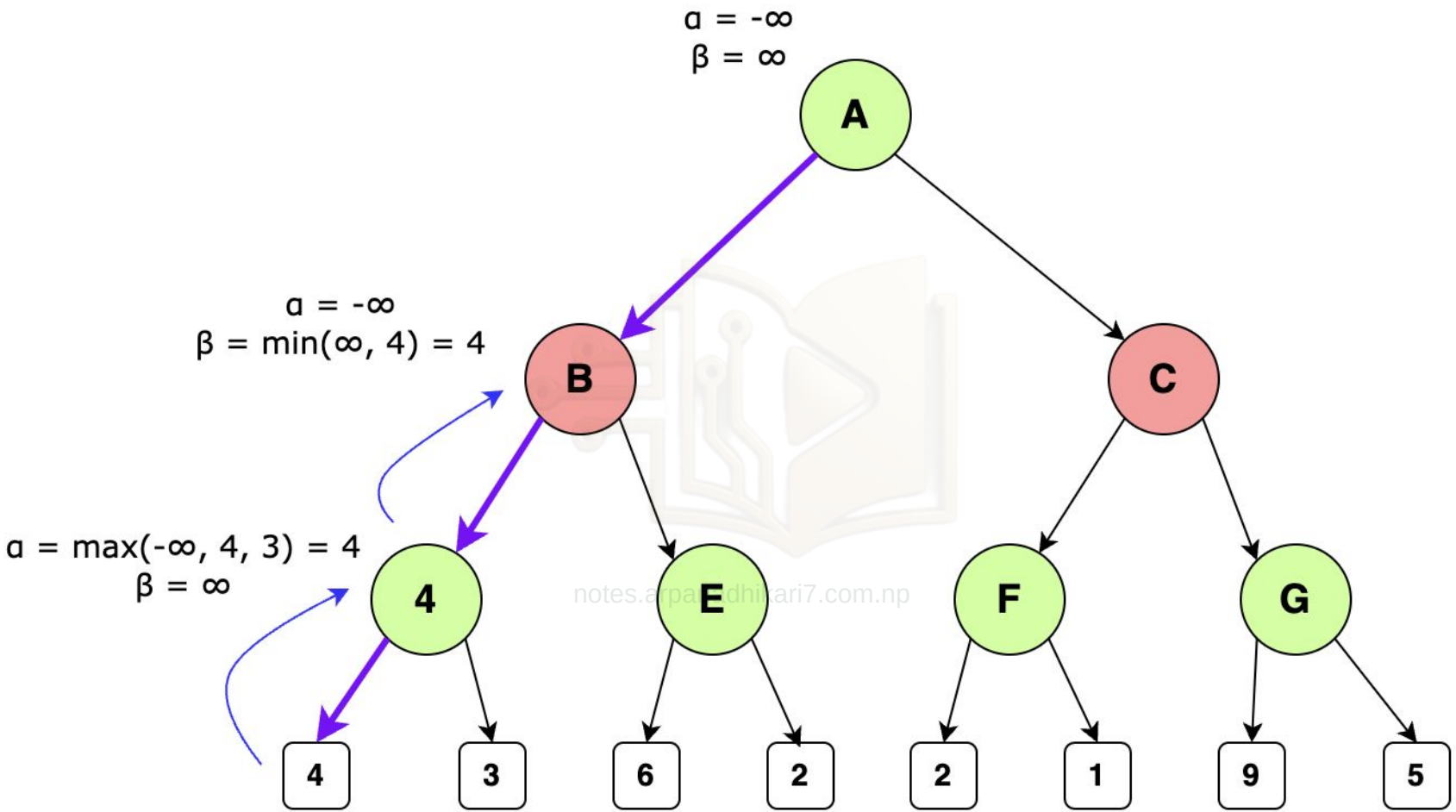
notes.arpanadhikari7.com.np

Alpha-beta search updates the values of alpha and beta as it goes along and prunes the remaining branches at a node as soon as the value of the current node is known to be worse than the current alpha or beta for MAX or MIN respectively.

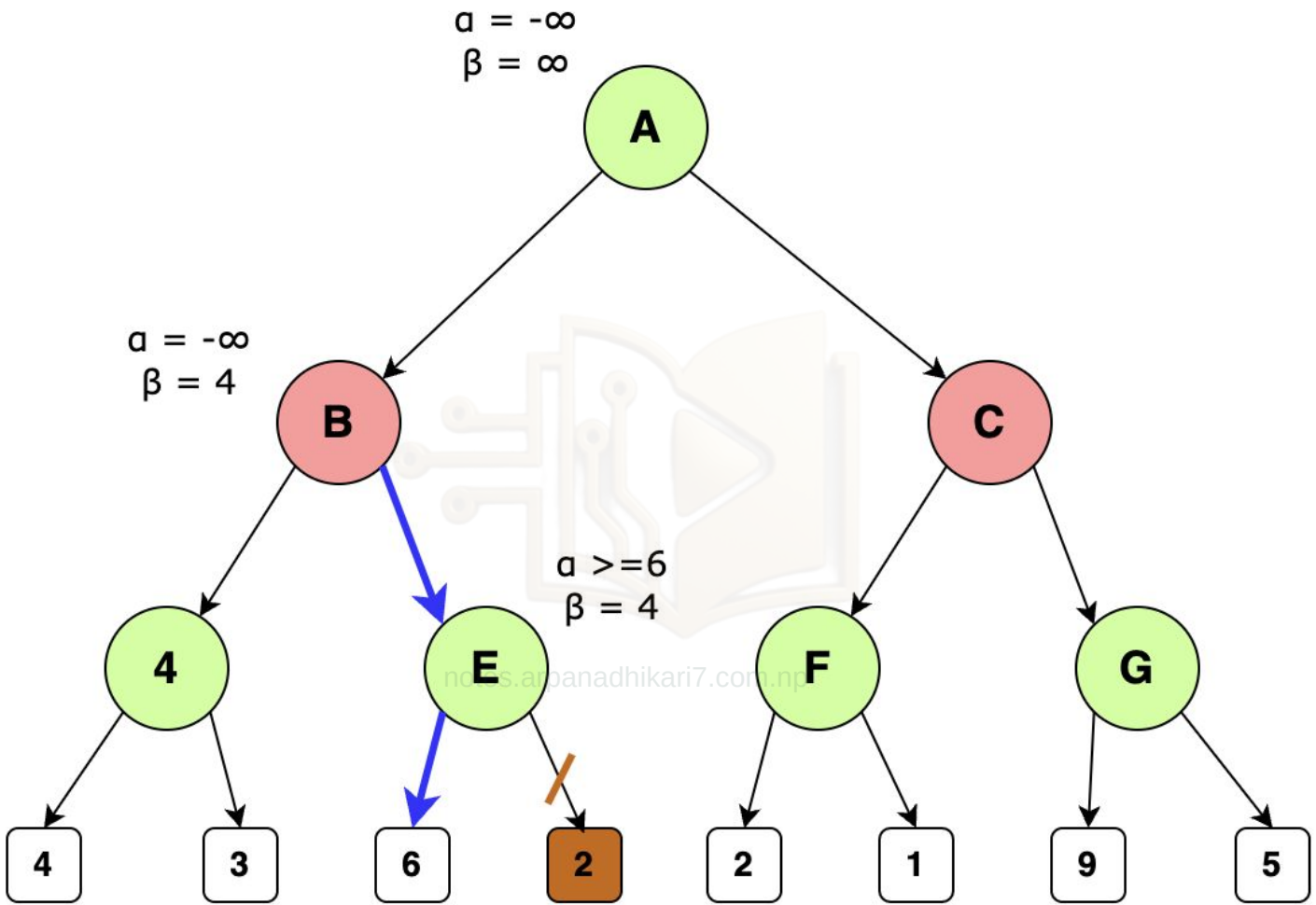
Available on notes.arpanadhikari7.com.np



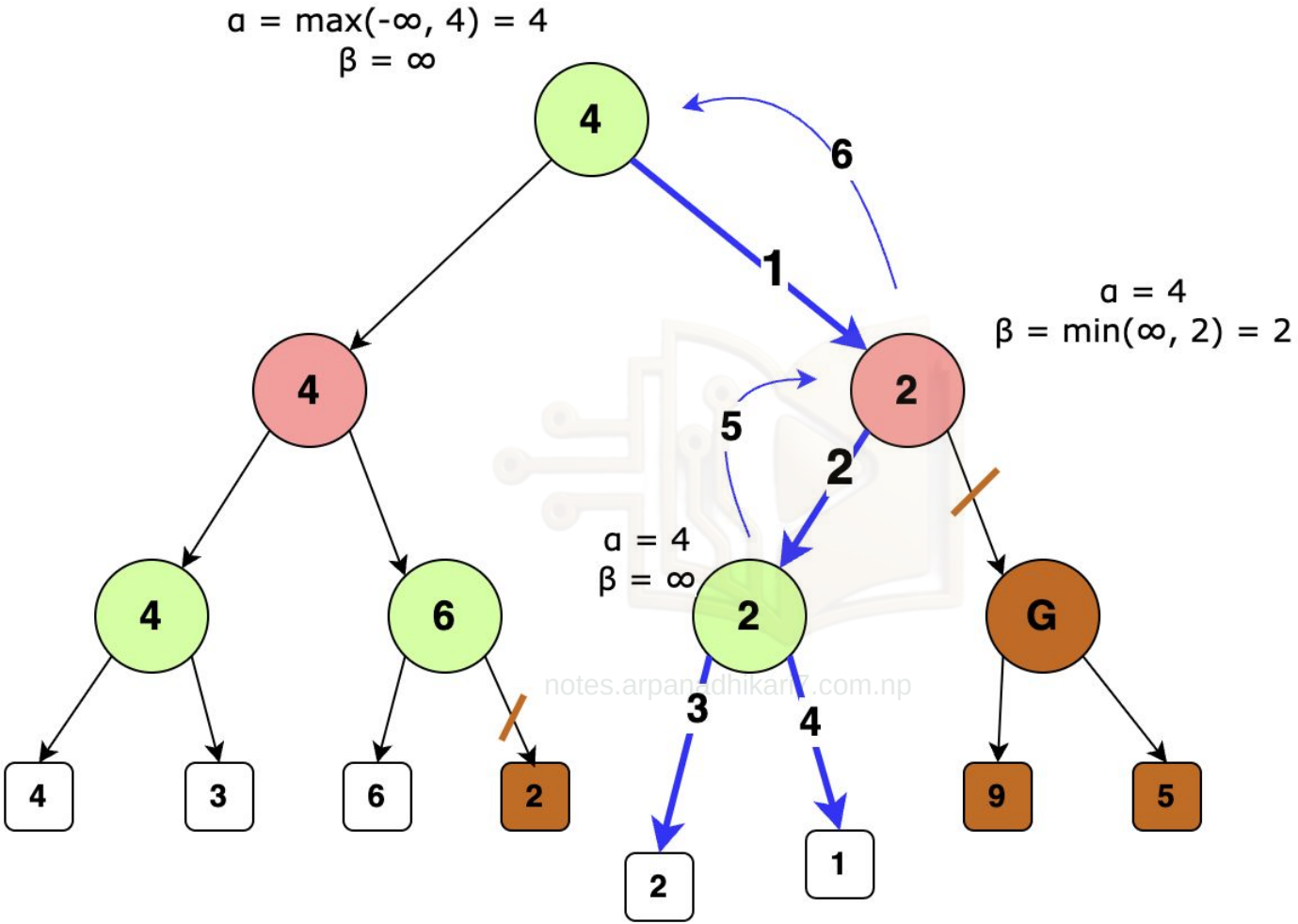

notes.arpanadhikari7.com.np




Available on notes.arpanadhikari7.com.np




Available on notes.arpanadhikari7.com.np




Available on notes.arpanadhikari7.com.np

# Example of Tic Tac Toe

There are two players denoted by X and O. They are alternatively writing their letter in one of the 9 cells of a 3 by 3 board. The winner is the one who succeeds in writing three letters in line.

The game begins with an empty board. It ends in a win for one player and a loss for the other, or possibly in a draw.

A complete tree is a representation of all the possible plays of the game. The root node is the initial state, in which it is the first player's turn to move (the player X).

The successors of the initial state are the states the player can reach in one move, their successors are the states resulting from the other player's possible replies, and so on.

Terminal states are those representing a win for X, loss for X, or a draw.

Each path from the root node to a terminal node gives a different complete play of the game. Figure given below shows the initial search space of Tic-Tac-Toe.

Available on notes.arpanadhikari7.com.np

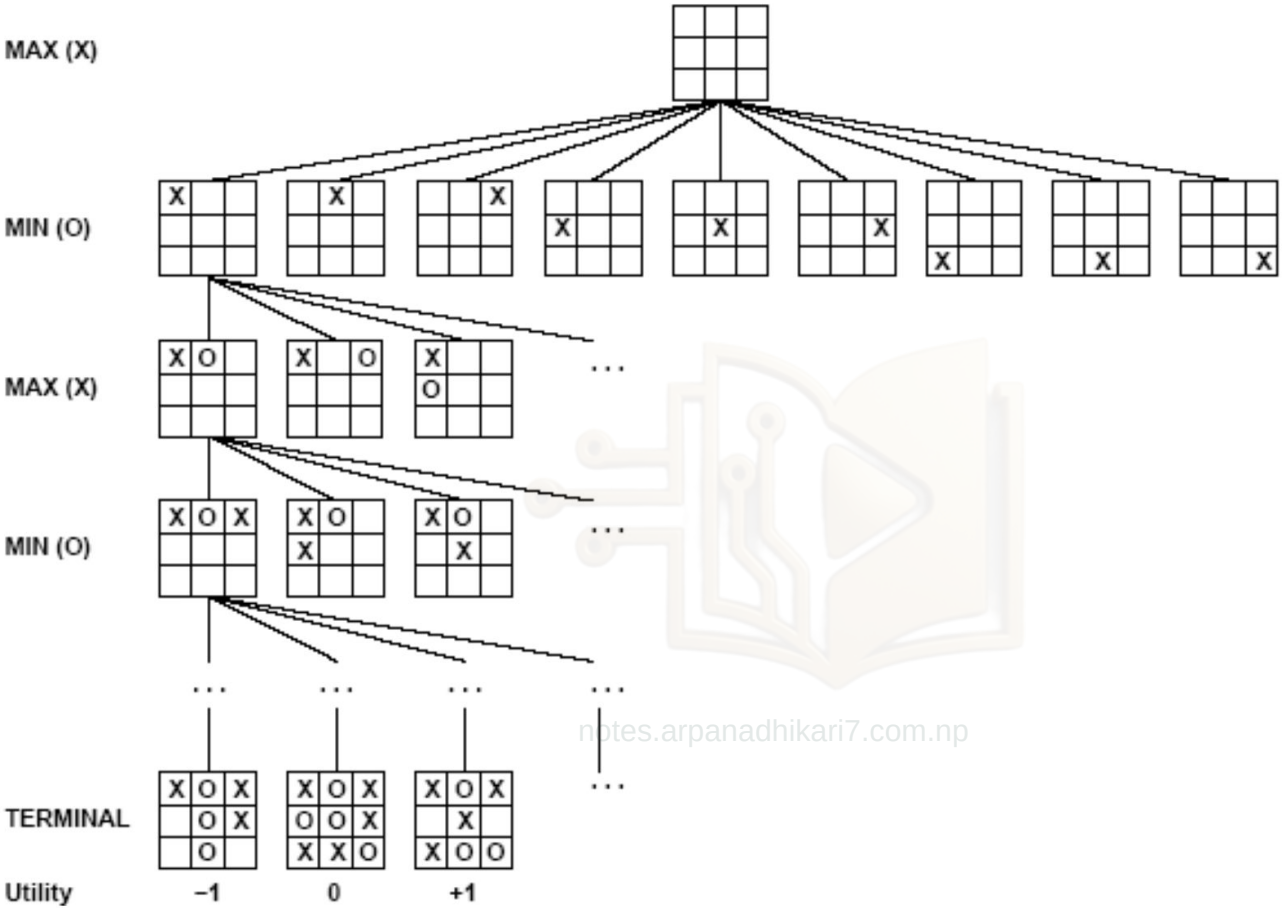



**Fig: Partial game tree for Tic-Tac-Toe**

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np